Technical University of Denmark

Quantifying Severeness of Adversarial Attacks

(02466) Project work

Celina Laungaard	s234809
Rune Harlyk	s234814
Lucas Mortensen	s234805
Christian Nickelsen	s234863

Abstract

Machine learning systems become increasingly integrated into high-stakes environments such as healthcare, autonomous vehicles, and cybersecurity. As a result, the threat posed by adversarial attacks through small, often imperceptible input perturbations that cause misclassification becomes more pressing. This project investigates the severity of three prominent adversarial attack methods: I-FGSM, PGD, and CW. Multiple metrics are introduced to quantify adversarialness, including KL-divergence, PSNR, SSIM, ERGAS, and attack success rate. These metrics are applied across a variety of model architectures with and without robustness strategies such as adversarial training, attention smoothing, and Kernel extension. Using the statistical test, mixed-effects modeling, and general data analysis, the attack's performance is compared across configurations, and the trade-offs between clean accuracy and adversarial robustness are assessed. Firstly, the results reveal a lacking performance for CW due to an incompatible experiment set up. Despite that, findings show a significant difference between all attack performances across all models. Additionally, both robustness strategies: 1) adversarial training and 2) architectural robustness changes, both effectively increase resistance against adversarial attacks as seen through a decrease in success rate and reduced KL-divergence, at the cost of PSNR and clean data accuracy.

June 23, 2025

Contents

1 Introduction

Artificial intelligence opens up for the possibility of more objective judgements and recommendations, not as plagued by human error, depending on training. It is increasingly incorporated into everyday use, both in the general public and in the workplace. The models currently used are still being developed, meaning that there still exists untested and unknown areas posing a potential risk. While this is often acceptable in low-risk applications, it becomes a significant concern when AI is introduced into high-stakes domains such as healthcare, defense, or autonomous systems [1].

The security of AI systems is still being developed and existing vulnerabilities undermine public trust in AI predictions. Uncertainties have no place in high-risk areas where mistakes can have huge consequences, especially where human lives are at risk. Relying on machine learning models to make decisions makes these areas particularly vulnerable to adversarial attacks. The application of adversarial attacks on AI is done with the intention of shifting the output through targeted or untargeted attacks, allowing malicious actors to force a desired outcome. Therefore, research into adversarial attacks is critical to ensure the security surrounding AI. Information is essential to become aware of any pitfalls and/or shortcomings of AI. By gaining knowledge on how adversarial attacks' severeness can be quantified and their effect on different models it can be used as a tool to create machine learning models that are robust and resilient against manipulation. Allowing for AI to be safely incorporated into high-risk fields while minimizing targeted attacks on human safety.

A key challenge regarding adversarial attacks lies in the quantification of their severity, as there are no standardized measurements. The purpose of this project is to implement different ways of measuring the severity of adversarial attacks and use these metrics to study the variation between different types of adversarial attacks. Through different methods of quantifying the adversarialness, the intent is to gain insight into the strengths and weaknesses of different attack types when applying them to both standard models and models with improved robustness. Through this, possible methods of limiting the severity of the attacks are assessed and knowledge is obtained regarding the consequences of the robustness methods on the model performance.

Three of the most studied white-box adversarial attacks are I-FGSM, PGD, and CW in which the adversary has full access to the model's parameters and gradients [2][3][4]. I-FGSM applies the Fast Gradient Sign Method repeatedly, producing a stronger perturbation than a single-step FGSM. PGD builds further on this idea by introducing random starts to create an even more potent iterative gradient-based attack. In contrast, the CW attack frames perturbation generation as a dedicated optimization problem as a slower but more powerful attack.

As previously stated, there are no standard measures of adversarialness for adversarial attacks, but common practices include attack success-rate (ASR), image quality assessments (IQA), and model confidence drops [5][6]. These methods are used to define the severity of the attack, which is necessary for developing techniques to increase a model's ability to oppose adversarial attacks. Within state-of-the-art robustness research, current examples of methods of increasing robustness of AI models are through training on adversarial examples, preprocessing and certified defenses [7][8]. A complementary method to these techniques are also improved architectural designs of the models [9]. Present challenges being researched in this field are specifically regarding the trade off between enhancing robustness without losing model accuracy [10][11]. Other problems include the transferability of adversarial examples between models, which would allow for attacks on black-box models [12].

The results of this project will be beneficial to anyone implementing image classifiers, as it provides perspective on the behavior of the most common adversarial attack types. As a lack of security is currently hindering the full capability of AI models, more research into adversarial attacks will create a better environment for reliance on AI systems.

To narrow and clearly define the focus of this project, the main objective has been formulated through three research questions and with a detailed description of how they are operationalized in this report:

1. How do the adversarial attacks I-FGSM, PGD, and CW vary in their severity and effectiveness in degrading the performance of machine learning models?
    - Create a range of adversarial examples. Conduct a systematic experiment applying each attack method to multiple models and measure their impact on performance using evaluation metrics. Statistically analyze the severity of each attack by comparing how much they each degrade model performance to determine if there is a significant difference between the adversarialness of the different types of attacks on different models.

2. How does the effectiveness and difficulty of improving robustness depend on the type and characteristics of the adversarial attack being performed?
    - Research, choose and implement methods that increase the robustness of the selected models. Use the metrics of adversarialness to calculate the adversarialness before and after finetuning the models. Analyze the results to determine any reduction in effectiveness of the adversarial attacks after implementation and discuss how the adversarial attacks are affected differently by robustness improvement methods.
3. Is there a trade-off between a model's accuracy on clean data and its robustness against adversarial attacks?
    - Train machine learning models with and without robustness-enhancing techniques and measure their performance on clean data. Subject the models to adversarial attacks and evaluate their performance degradation under attack. Compare accuracy on clean data versus adversarial robustness to identify potential trade-offs. Examine the pros and cons of this trade-off, considering practical applications, model generalization, and real-world deployment.

This report will apply theoretical frameworks related to adversarial attacks, robustness, IQA methods, and evaluation metrics, along with dataset management, large-scale pipelines, and data analysis, to address these questions. The pipeline will download and downsize a dataset, generate a variety of adversarial attacks, and analyze these attacks for their severity using different IQAs. Baseline models are compared against their robust counterparts as well as adversarially trained versions. The results were analyzed using methods including mixed effects method across the different attack types to observe their severity.

The report begins with a detailed examination of the method and theory behind the experiment, where the pipeline and its building blocks are explained. It continues with a presentation of the gathered data and a result section presenting the analysis of the collected data. Lastly, it finishes of with a discussion of the presented results and the general execution of the experiment with sources of error, points of reflections, and future expansions and considerations.

2 Method

2.1 Adversarial Attack Types

Adversarial attacks are already being explored because of the threat they pose to AI models. The attack methods fall under the category of being white box or black box attacks. White box attacks are defined by attackers having full access to the AI model architecture, while black box attacks are defined as having access to only the input and output of the AI model. The types of attacks can further be split into the categories "Poisoning", "Evasion", "Model extraction", and "Inference".

Poisoning is a white-box attack that works by affecting the training data the AI model uses. By poisoning the training data, you can cause the AI model to behave mischievously or perform worse. Evasion is a white-box attack that works by deceiving already trained AIs or classifiers. The deceiving input is usually invisible to the naked eye, but the model will be affected. Model extraction is a black-box attack that attempts to learn the structure or architecture of a model to recreate it [13]. In this project, the focus is on the "evasion" attacks I-FGSM, PGD, and CW in a white-box environment.

The chosen attacks were configured to effortlessly switch between attack methods in the pipeline. Furthermore, the attacks were optimized for batching in order to take in and process multiple images at a time. Each attack returns a list with values for the following categories for each image:

- Perturbed image
- Successful attack - True/False
- First successful iteration

2.1.1 Gradient-based attack types

Adversarial examples are images where a certain amount of perturbation has been applied in order to cause a classifier to misclassify the image. These perturbed images only vary minimally from the correctly classified images that they are based on, which makes the adversarial example so similar to the original image that they are indistinguishable to the human eye.

The reason behind adversarial images being able to fool deep networks for adversarial attack types such as I-FGSM and PGD can be explained linearly. Digital images are most often represented with 8-bits per color per pixel, meaning they discard all information below 1/255 of the dynamic range. These are cast to floating-point tensors, so perturbations with a magnitude below 1/255 survive. Because deep nets behave (locally) like a linear mapping in input space, for a perturbed image

$$\tilde{x} = x + \eta$$

we have

$$w^T \tilde{x} = w^T \cdot x + w^T \cdot \eta$$

where w is the weight or gradient of the vector at that point. Iterative attacks like I-FGSM and PGD exploit this by taking repeated small gradient steps, accumulating a perturbation until the network's prediction flips [14][3].

2.1.2 Iterative Fast Gradient Sign Method

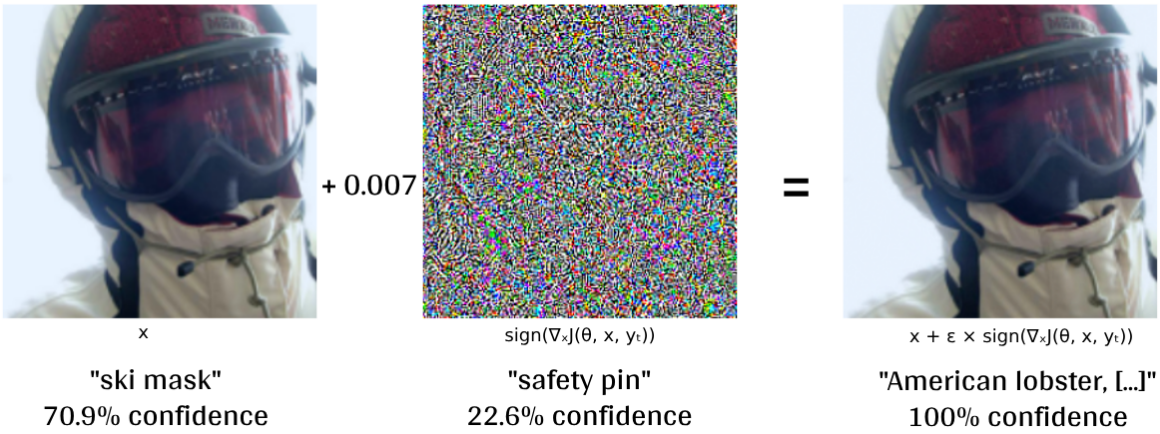


Figure 1: *Example of I-FGSM; showing original image (on the left), classified as "ski mask", the noise added, whereof the image becomes classified as "American lobster, [...]"*[14]

The "fast gradient sign method" (FGSM) is created by Ian J. Goodfellow, et al. in the paper "EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES". FGSM creates adversarial images and its working method is demonstrated in figure 1 above. θ is the parameters of a model, x is the input, y is the target, and $J(\theta, x, y)$ is the loss function used to train the neural network. By linearizing the loss function with respect to the input x, FGSM computes the optimal perturbation under an l_∞ (max-norm) constraint as:

$$\eta = \epsilon \text{sign}(\nabla_x J(\theta, x, y))$$

The adversarial example is then constructed by adding this perturbation to the original input [14]:

$$x_{adv} = x + \eta$$

Iterative FGSM (I-FGSM), also known as Basic Iterative Method or BIM, refines FGSM by applying it over multiple smaller steps and clamping the per pixel value to be in the ϵ-neighborhood [2].

$$x_0^{\text{adv}} = x, \quad x_{n+1}^{\text{adv}} = x \; + \; \Pi_x\Big(x_n^{\text{adv}} + \epsilon \, \text{sign}\big(\nabla_x J(\theta, \, x_n^{\text{adv}}, \, y)\big) - x\Big) \quad n = 0, \ldots, N-1$$

where N is the number of steps. This project sat $\epsilon = 4/255 \approx 0.016$ and $N = 100$, following common practice in the adversarial robustness literature [15].

2.1.3 Project Gradient Descent

Project Gradient Descent (PGD) is a continuation of I-FGSM, developed by A. Madry et al. published in 2017. PGD utilizes a uniform random perturbation start. It is described with the following equation:

$$x^{t+1} = \Pi_{x+\mathcal{S}}(x^t + \alpha sign(\nabla_x J(\theta, x, y)))$$

Here x is the perturbed image at the current iteration t, α is the step size, $J(\theta, x, y)$ is the gradient of the loss function, θ is parameters of a model, $\mathcal{S}$ is a uniform random perturbation start, Π is the projection operator which ensures the perturbed images stays within a valid range [3].

For PGD we expand the parameters from I-FGSM and set $\alpha = \epsilon/N \approx 0.00016$ so that the full perturbation budget ϵ is evenly distributed over the N iterations [2].

2.1.4 Carlini and Wagner

Carlini and Wagner attack (CW) is an adversarial attack developed by Nicholas Carlini and David Wagner presented in their report "Towards Evaluating the Robustness of Neural Networks" from 2017. In this project, the L_2 version of the attack with Euclidean distance measure is used, which attempts to minimize the attack and keep the perturbed image as close as possible to the original one. It is formulated as follows:

$$\text{minimize}||\delta||_p + c \cdot f(x+\delta)$$

f is the loss function, $||\delta||_p$ is the p-norm of the perturbation. $x+\delta \in [0,1]^n$, so the adversarial image is clamped in a normalized range. This part of the formulation is achieved by calculating the distance metric between the original image and the perturbed image, leaving only the perturbation. x is the original image. c is a constant that controls the trade-off between the two terms and therefore balances small perturbations and achieving misclassification. For this report, we fixed the value of c, but it is commonly optimized.

The following objective function is used:

$$\text{minimize}||\frac{1}{2}(\tanh(w)+1) - x||_2^2 + c \cdot f(\frac{1}{2}\tanh(w)+1)$$

Where w is the adversarial image. The attack uses gradients to target a specific class and Adam optimizer to ensure minimized image perturbation while achieving misclassifications. The tanh-function is used to keep the image valid and handle box constraints [16].

The following parameters were chosen for CW: $lr = 0.01$ [17] as standard in most implementation, $N = 100$ allows for successful attacks while budgeting computation, $c = 10$ and $\kappa = 15$ were selected through experimentation.

2.2 Dataset [ImageNet]

2.2.1 ImageNet20

ImageNet is a large-scale visual database, which includes 22000 classes spread across 14 million images. A subset of this dataset is the ILSVRC version, which stands for ImageNet Large Scale Visual Recognition Challenge. This version is a carefully curated and balanced subset of ImageNet and contains approximately 1.3 million labeled images distributed across 1000 object categories [18][19]. Each class corresponds to a fine-grained category, often reflecting specific object types or species. For instance, classes include (tench, Tinca tinca), (goldfish, Carassius auratus), and (buckeye, horse chestnut, conker). Illustrating ImageNet's emphasis on detailed semantic distinctions rather than broad categories like "fish" or "tree". The dataset is widely used for training and evaluating computer vision models, and its influence has been pivotal in the development of deep learning architectures such as AlexNet, ResNet, and EfficientNet, and is still used in many state-of-the-art benchmarks.

While the full ImageNet dataset is composed of high-resolution images originally collected from the internet, the images used in this project are resized and center-cropped to 224×224 pixels as is standard practice, making them compatible with the project's model input requirements.

Due to the size of ImageNet, we constructed **ImageNet20**, a balanced subset of twenty classes chosen to span five broad semantic categories—animals, plants, tools, vehicles, and household objects. Within each category, we picked examples that differ in color (e.g. red fox vs. cauliflower), and texture or shape (e.g. gas mask vs. slide rule). This subset allows us to probe adversarial transfer across a wide range of visual concepts while keeping computational costs manageable. Due to the sampling strategy, the resulting dataset had some minor class imbalances, where of 3 classes were slightly underrepresented.

1. green mamba	11. theater curtain, theatre curtain
2. Doberman, Doberman pinscher	12. red fox, Vulpes vulpes
3. cocktail shaker	13. slide rule, slipstick
4. garter snake, grass snake	14. walking stick, walkingstick, stick insect
5. pineapple, ananas	15. obelisk
6. American lobster, Northern lobster, Maine lobster	16. harmonica, mouth organ, harp, mouth harp
7. ambulance	17. mousetrap
8. cauliflower	18. ski mask
9. pirate, pirate ship	19. laptop, laptop computer
10. safety pin	20. gasmask, respirator, gas helmet

Table 1: Classes for **ImageNet20**

By using our ImageNet20 classes, we filter the dataset and cache the indices from a larger dataset (Link to parent dataset is found in appendixA.2 [20]). This approach accelerates data loading and guarantees reproducible experiments.

Furthermore, the dataset is split into train, validation, and test sets using fixed ratios - 70/15/15 with a fixed seed to ensure reproducibility. This roughly translates to 18000 images for training, 4000 for validating, and 4000 for testing. These splits are saved in a .pkl cache file and reloaded during runs to maintain consistent evaluation. Each image is also preprocessed using standard ImageNet transformations tailored for either training or evaluation. This setup is wrapped in dataloaders for efficient batching during model training and testing.

2.2.2 Privacy and Bias

The ImageNet dataset contains class labels that inevitably include people such as "Scuba diver", "basketball player", "groom", and "firefighter". Even images where people are not necessarily needed might include them, such as "ski mask" and "bonnet". People could also just appear in the background of all classes. This opens up for two main concerns in ethics regarding privacy and bias. Including pictures of people opens up for potentially violating privacy rights as the pictures has to be ethically sourced with permissions from the included people. Additionally, the images also has to be representative of the class they are representing for categories such as for the "basketball player" and "doctor" because the human here is an undeniable part of the classification and the model will have to be trained on both genders and all races to reduce bias.

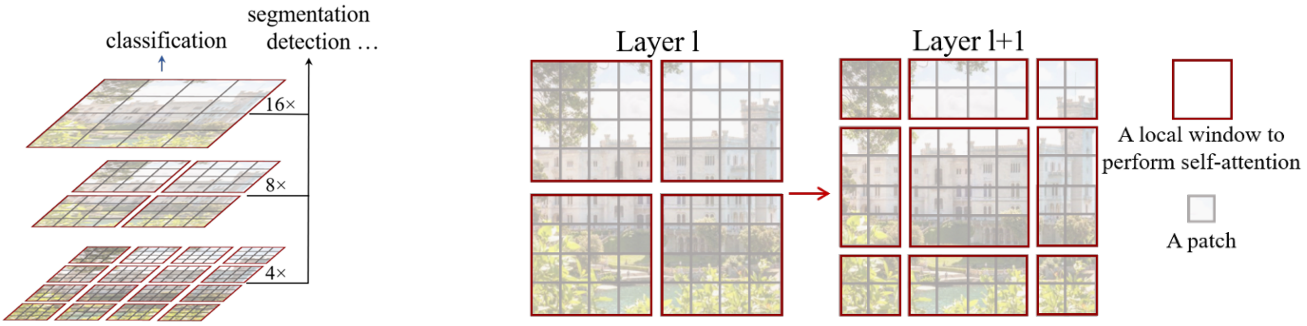
2.3 Selected Models

Three pre-trained image classifiers were selected for the evaluation of the attacks. This choice was made to reduce computational cost while maintaining high baseline accuracy. However, due to using a subset of ImageNet with 20 classes, it was necessary to finetune the head of each model. Pretrained weights were loaded for each model and retrained only for the head to achieve better accuracy and limit computational load. Two widely recognized CNNs and a Transformer were selected to see whether different model architectures might be less or more responsive to adversarial attacks.

2.3.1 Swin Transformer

Swin Transformer (Shifted Window Transformer) is a model created by Ze Liu et al. presented in the report "*Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*" [21] in 2021. It is built using transformers, broadly known from their use in LLMs. Previously, the problem with implementing transformers in vision applications was the difference in scale and dimension compared to language, as visual elements can vary substantially more and often require dense prediction at the pixel level, which would be intractable for a Transformer on high-resolution images.

The Swin transformer creates a hierarchical representation by separating the image into smaller patches. These are gradually merged with neighboring patches in the deeper layers to create bigger patches, allowing it to capture both smaller and larger objects. To solve the problem of dimensionality, the attention mechanism is only applied independently of each other to these patches, reducing the complexity from being quadratic to linear [21].



(a) The hierarchical representation with patch merging reduces spatial dimensions while increasing semantic abstraction, enabling tasks such as classification, segmentation, and detection.

(b) The shifted window mechanism alternates between layers: self-attention is computed within non-overlapping local windows in one layer, and the window configuration is shifted in the next layer to enable cross-window connections and improve modeling capacity.

Figure 2: Illustration of the SWIN Transformer architecture [21]

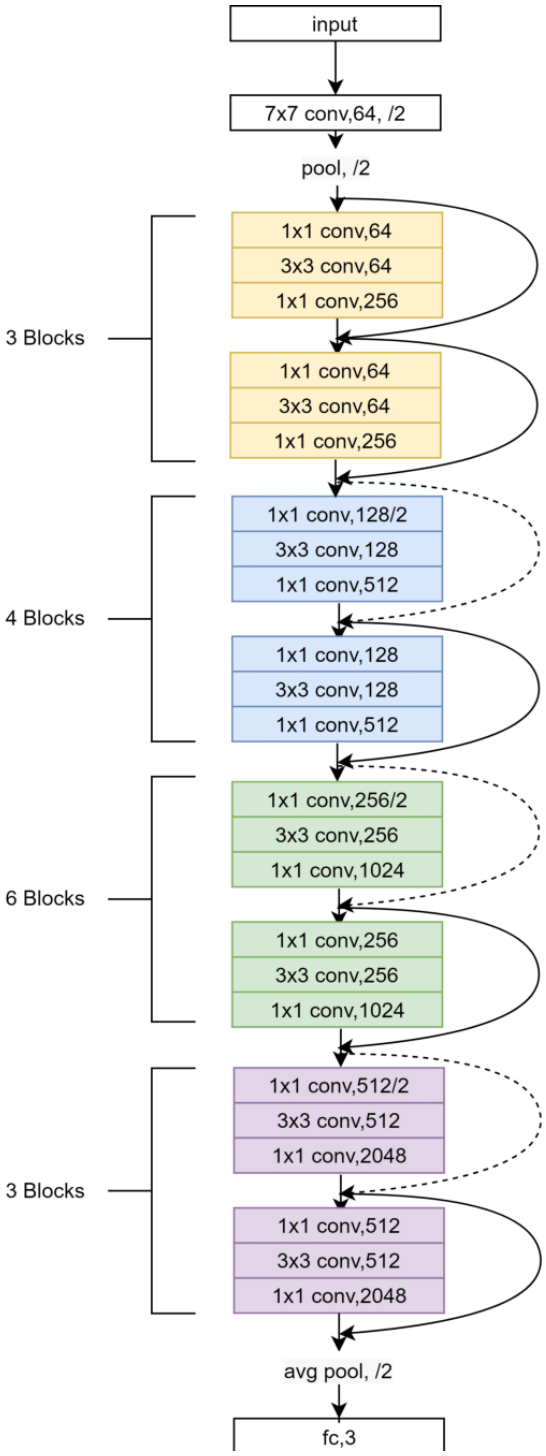
A key element of the Swin transformer is the shifting of window partitions between self-attention layers. The overlaying of the current windows with the windows from the preceding layer allows for connections among them that significantly enhance modeling power. Swin reached a top-1 accuracy of 81.5% on ImageNet-1K [21].

2.3.2 ResNet-50

ResNet was introduced in the paper "*Deep Residual Learning for Image Recognition*" released in 2015 by K. He et al. [22]. It is a residual network, created by Microsoft Asia Team in 2015, that makes use of residual learning. To overcome the challenge of vanishing gradients and degradation of the training accuracy, each layer learns a residual function instead of a complete transformation. In this project, ResNet-50 was selected, meaning it is a 50-layer deep convolutional neural network. The model can take input images of size 224 by 224, which fits our intended ImageNet dataset. ResNet-50 reached a top-1 accuracy of 76.1% on ImageNet-1K.

2.3.3 MobileNet-V3

Mobilenet was presented in the paper "*MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*" by A.G. Howard et al. in 2017 [23]. MobileNet was originally created with the purpose of being a lightweight, efficient and fast neural network, which could be implemented in mobile devices and embedded systems. It allows for selecting latency based on resource restrictions while minimizing accuracy loss. The key element of MobileNet is the use of depthwise separable convolutions, where it separate the convolution into two steps: filtering and combining. This substantially reduces the parameters that have to be learned, with the effect of drastically reducing computations and model size. Like with the previous two models, the head of the model is updated to output class labels of dimension 20 to fit with the reduced ImageNet. MobileNet reached a top-1 accuracy of 74.0 % on ImageNet-1K.



(a) Illustration of ResNet-50 architecture [24]

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

(b) Standard MobileNet architectural structure [23]

Figure 3: Comparison of ResNet-50 vs. MobileNet backbone structures

2.4 Robustness against Attacks

Considering the attack types, it is clear that the gradient and loss functions are the most important aspects of them. Therefore, to improve robustness against attacks would be to manipulate the landscape of the loss functions. Smoothing the loss function is a well-documented method for increasing robustness [25]. Therefore, the implemented methods for improving model robustness will focus on smoothing the loss function and/or creating less focus on individual pixels.

There are many ways to improve the defense against adversarial attacks. Two of the most popular methods are 1) adversarial training, where the model is trained on adversarial data, and 2) architectural improvement, where the architecture of the model is changed to better withstand adversarial attacks. These methods will throughout the project be referred to as ATM and REAM respectively. It has become widely known that certain changes to the architecture and training on adversarial data will lead to models becoming harder to fool using adversarial attacks. In this report, the following robustness techniques will be implemented for the three models:

ResNet-50	Standard	Adversarial training	Squeeze & Excitation
MobileNet	Standard	Adversarial training	Width extensions + stem kernel increase
Swin	Standard	Adversarial training	Attention smoothing

Table 2: The 9 different models implemented in the project

Table 2 presents the 9 different models examined in the project. The three base models in the left column with standard architecture and default weights from PyTorch (except finetuned head). The three adversarial models based on the three default models, but fine-tuned with adversarial training. The three robust models based on the default models trained from the bottom with robust architectural changes. The methods used for the architectural changes are Squeeze and Excitation, Width Extension, Stem Kernel increase and Attention Smoothing.

2.4.1 Adversarial training [All models]

For adversarial training, the models were trained for 5 epochs with an 80 % chance of having the input batch be transformed to adversarial images. The attack method was randomly selected between I-FGSM and PGD for each batch. This setup allows the models to keep their training knowledge of the original images while learning how to mitigate the effect of the adversarial attacks. I-FGSM and PGD were selected as they are much faster than cw, but still provide valuable insights.

2.4.2 Squeeze and Excitation [ResNet]

Squeeze and Excitation network (SENet), originally researched by J. Hu et al. in the paper "Squeeze-and-Excitation Networks", is a building block that can be implemented in CNNs [26]. It is computationally lightweight and helps the model focus on informative features, suppressing uninformative features and noise. It works like a dynamic recalibrator, ensuring features in the network that are interesting are focused on. These feature weights for interest constantly vary and work dynamically for each image analyzed.

The Squeeze part of the SENet works by squeezing global spatial information into a channel descriptor using global average pooling. Having 50 channels, the end result will be a vector of 50 values from the global average pool [26].

The formula for squeeze is given by:

$$\mathbf{F}_{sq}(\mathbf{u}_c) = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j)$$

Where u_c is the output of channel c, H is the height of the spatial dimensions of a feature map and W is its width.

The Excitation part works by taking this vector and passing it through a fully connected layer, a ReLU layer, another fully connected layer and finally a sigmoid function. This series changes the dimensions multiple times to ensure a generalized understanding and relationships between channels. The entire procedure ends with a vector of weights between 0 and 1 with a length corresponding to the number of channels [26].

The Excitation part can be described by:

$$F_{ex} = \sigma(\mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{F}_{\mathbf{sq}}))$$

Where σ is a sigmoid function, δ is a ReLU function. The two W's are weights for the fully connected layers. In the article, they use z and $\mathbf{F}_{sq}$ for the squeeze function, but for simplicity only the latter is used [26].

2.4.3 Width extensions [MobileNet]

From general knowledge of neural networks, the meaning of "widening" the network means adding more filters to each layer, leading to the model learning a wider variety of features from the input data. Generally, a greater knowledge of features leads to a decrease in the success of adversarial attacks [27] [28].

2.4.4 Stem Kernel increase [MobileNet]

Some research suggests that increasing the size of the stem or first layer of the CNN can help with robustness, as it can act as a filter of noise [29]. In the cited paper "First line of defense: A robust first layer mitigates adversarial attacks", they also use maxpooling features that we left out of MobileNet REAM. Other researchers are convinced that generally larger kernels in a CNN are more robust[30].

2.4.5 Attention Smoothing [Swin]

The approach used in this project to improve robustness for Swin is a method called attention smoothing [31]. It is a version of temperature scaling ensuring the models focus is on the correct items.

The Swin model will, while analyzing an image, output attention logits; however, in the robust model, these attention logits will have temperature scaling applied to ensure the attention weights are smoothed and won't weight a couple of tokens highly. [32]. It will be happening directly in the model.

This change will have several benefits, including improved training stability, better model calibration, and more robust features. The first and second are explained by the common issue of transformers being too focused and experiencing "attention entropy collapse". The constant and deeply integrated temperature scaling will ensure the focus does not get locked in on single tokens, also avoiding the overconfident internal representations often seen [33] [34] [35]. The third point is a result of the first two improvements. The broader view on all features will result in reduced influence of noise from attacks making it less susceptible to misclassifications [36].

2.5 Image Quality Assessment methods

Image Quality Assessments (IQA) are used to quantify the similarity of images. It's commonly used to determine the loss-of-quality for compression methods, either for streaming, file compression, restoration etc. This means they can be used to quantify the difference between images. They come in handy as it provides a quantifiable method for comparing the impact an attack has on an image's quality/similarity in relation to the original image. Most common use is with full-reference methods, which expects both images (source and perturbed image in our case). All the following methods are full-reference.

2.5.1 Peak Signal to Noise Ratio

Peak Signal to Noise Ratio (PSNR) is a widely used metric for image quality degradation. It compares the maximum possible signal power to noise power or "corruption" noise [37]. It is widely used to evaluate noise-reduction methods and compression algorithms. PSNR is expressed in decibels where higher values indicate better image fidelity. [38] The mathematics behind the method is:

$$\text{PSNR} = 10log_{10}\frac{R^2}{MSE}$$

Where R is the maximum possible value (power) of the signal (for 8-bit images, $R = 255$) and MSE is the mean squared error with the equation:

$$MSE = \frac{\sum_{M,N}[I_1(m,n) - I_2(m,n)]^2}{M \cdot N}$$

Where I_1 is the original and I_2 is the perturbed image. m and n describe the number of rows and columns [39].

2.5.2 Structural Similarity Index Measure

The main focus of structural Similarity Index Measure or SSIM is preservation of structural information between two images with the goal of trying to mimic the human visual system. The human visual system is highly adapted to extract structural information from the viewing field. As such, a measure of structural information change provides an approximation to perceive image distortion [40].

Compared to other IQA methods, such as MSE and PSNR, which evaluate the pixels individually, SSIM compares local patterns of pixel intensities that have been normalized for luminance and contrast. Pixels in an image are

highly related to their neighboring pixels and many methods look at isolated pictures; but even when using linear transformation to decompose the images, it does not fully remove this correlation. SSIM instead proposes a more direct measure of the structural difference between the original and perturbed image[41].

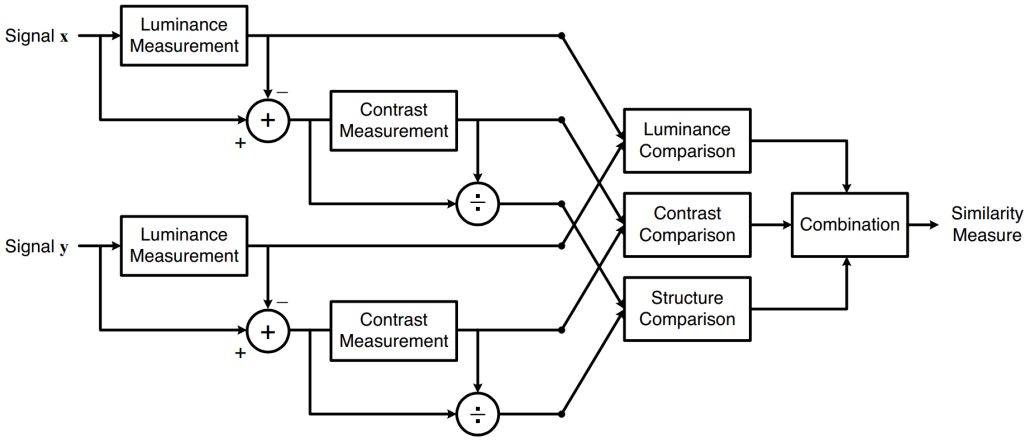


Fig. 3. Diagram of the structural similarity (SSIM) measurement system.

Figure 4: Diagram of SSIM [42]

SSIM is built around 3 comparisons of luminance, contrast, and structure, which are combined into one similarity measure[40]:

$$\text{SSIM}(x,y) = f(l(x,y), c(x,y), s(x,y)) = \frac{(2\mu_{\mathbf{A}}\mu_{\mathbf{B}} + C_1)(2\sigma_{\mathbf{AB}} + C_2)}{(\mu_{\mathbf{A}}^2 + \mu_{\mathbf{B}}^2 + C_1)(\sigma_{\mathbf{A}}^2 + \sigma_{\mathbf{B}}^2 + C_2)}$$

Where $\mathbf{A}$ and $\mathbf{B}$ are the input images. $\mu_{\mathbf{A}}$ and $\mu_{\mathbf{B}}$ is the mean for the given input image in a window around pixel value (x,y). $\sigma_{\mathbf{A}}$, $\sigma_{\mathbf{B}}$ and $\sigma_{\mathbf{AB}}$ are the variance and covariance in the window. C_1 and C_2 are values added to avoid division by zero. We set the values to $C_1 = (0.01 \cdot 255)^2$ and $C_2 = (0.03 \cdot 255)^2$. The formula is derived through f that uses the following formulas describing luminance $l(x,y)$, contrast $c(x,y)$ and structure $s(x,y)$.

$$l(x,y) = \frac{2\mu_{\mathbf{A}}\mu_{\mathbf{B}} + C_1}{\mu_{\mathbf{A}}^2 + \mu_{\mathbf{B}}^2 + C_1}$$

$$c(x,y) = \frac{2\sigma_{\mathbf{A}}\sigma_{\mathbf{B}} + C_2}{\sigma_{\mathbf{A}}^2 + \sigma_{\mathbf{B}}^2 + C_2}$$

$$s(x,y) = \frac{\sigma_{\mathbf{AB}} + C_3}{\sigma_{\mathbf{A}}\sigma_{\mathbf{B}} + C_3}$$

C_3 being a constant to avoid division by zero, it was proposed by Wang et al. to set it equal to $C_3 = C_2/2$ which results in the states SSIM formula.

2.5.3 Error relative global dimensionless synthesis

Error relative global dimensionless synthesis (ERGAS) is usually used to calculate the accuracy of pan-sharpened images [43]. For this report, it calculates how well the color accuracy of the original image has been preserved in the perturbed image.

It uses the function:

$$ERGAS = \frac{100}{r} \cdot \sqrt{\frac{1}{N}\sum_{k=1}^{N} \frac{RMSE(B_k)}{\mu_k^2}}$$

Where r denotes pixel size between images, N is the number of spectral bands, $RMSE(B_k)$ is the root mean squared error of band k between images, and μ_k^2 is the mean value of band k of the reference image [44].

2.6 Evaluation metrics

A crucial part of the pipeline and experiment is the metrics of which the adversarialness of attacks are evaluated. There is currently no universally accepted method for quantifying the adversarialness of adversarial attacks. In this project, three complementary metrics are used to compare the severity and effectiveness of the three attack types, described in the following sections:

2.6.1 Attack Success Rate

A basic method of evaluating the adversarial attacks is calculating and comparing the attack success rate (ASR), which is determined to be the proportion of samples for which the model is successfully fooled [6]. This measurement can be further expanded by considering the level of perturbation they apply to the image. An attack with a high success rate but large perturbations may be less desirable than one with moderate success but minimal visual distortion.

2.6.2 Image Quality Assessments

It's a logical train of thought to look at and compare image quality before and after perturbation is applied, as adversarial attacks are characterized as being supposedly undetectable to the naked eye. The Image Quality Assessment (IQA) methods described earlier provide a quantitative estimate of the perturbation severity. In combination with the ASR, it can be used to access the effectiveness of the attack types, as they might have a high ASR, but the IQA methods might reveal the level of perturbation needed to achieve that level of success. This helps give insight into methods' inner mechanics and differentiate between attacks that subtly manipulate the input versus those that rely on more aggressive perturbations.

2.6.3 Difference in probability distribution

The goal of the attacks is to shift the probability distribution enough to cause the model to misclassify the image. Both the CNN's and transformer-based model used in this project output a full probability distribution over all possible classes. This opens up for comparing the probability distribution shifts between the predictions before and after adding perturbation to the source images. The KL-divergence is computed between the two probability distributions to measure the change. A mixed model test is applied to the KL-divergence value for each sample to compare the impact of the three attack types on the probability distribution.

2.7 Model, attack and data setup:

The experiment was run through a pipeline, which is built on multiple, separate building blocks such as the dataset, configuration files, the models, and the adversarial attacks. To streamline the process and allow us to only need one pipeline, we constructed all elements in modules to keep the pipeline modular, adaptable, and manageable.

To ensure reproducibility across different runs, Anaconda was utilized to manage the environment, packages, and their versions. Python 3.11 was chosen as it allows some of the newest PyTorch features with CUDA-compatibility. Furthermore, the pipeline was seeded, and all relevant flags for the code to run deterministic were defined. To simplify this process, a function *set_seed* was introduced. This included seeding the modules `random`, `numpy-random`, `torch-random` and `torch-cuda`. Clear instructions for recreating the environment is included in the README.me found in the project repository located in appendix A.1.

The model training and pipeline was run on the following device configuration:

- OS: Windows 11 + Linux on the HPC
- Local Hardware: Intel chips and Nvidia graphics cards
- HPC Hardware: A100 GPU and V100 GPU `https://www.hpc.dtu.dk/`

2.8 Pipeline structure

With the modules described in the previous sections, the complete pipeline was constructed to simplify the process of generating adversarial examples and evaluating them across models. For each combination of model and attack, the pipeline generates a specified number of adversarial examples. Given a configuration, the total number of adversarial examples generated is calculated:

$$n \cdot c \cdot m \cdot a$$

where n is the number of images from each class, c is the number of classes, m is the number of models and a is number of attacks.

The pipeline streamlines the execution of attacks. After completing model training (base, ATM, and REAM), attacks are launched by configuring the pipeline. It takes a number of arguments that define what models, attacks, hyperparameters etc. to use. This setup limits errors and helps streamline attacks across different devices. It defines a seeded target class for each image in the test split, batches these combinations, and reuses them across all attack and model combinations for consistent comparisons. Each combination is loaded and run through all batches, logging evaluation metrics for each image. Once completed, the process is repeated for the next combination. All results is saved both locally and to Weights & Biases. Locally a batch size of 32 was used, while the HPC allow for a significant larger size of over 256 depending on the available resources.

The high-level structure of the pipeline can be seen below.

Algorithm 1 Adversarial Generation Pipeline

Require: GenerationConfig cfg
1: $batches \leftarrow$ PreprocessBatches(cfg)
2: **for all** m **in** $cfg.models$ **do**
3: $model \leftarrow$ LoadModel(m)
4: **for all** a **in** $cfg.attacks$ **do**
5: **for all** b **in** $batches$ **do**
6: $gen\_cfg \leftarrow$ BuildConfig(cfg, m, a, b)
7: $res \leftarrow$ RunSingleGeneration(gen_cfg)
8: LogAndStore(res)
9: **end for**
10: **end for**
11: **end for**
12: Summarize(cfg)

Algorithm 2 RunSingleGeneration

Require: AttackConfig cfg_{attack}
1: Set deterministic seed
2: Select device
3: Prepare input batch
4: Evaluate model on clean inputs
5: Generate adversarial examples
6: Evaluate model on adversarial inputs
7: Compute distortion metrics
8: Save adversarial images
9: Assemble and return results

There is a large computational cost associated with creating adversarial images and evaluating them on the scale required and therefore necessary optimizations was implemented. This includes: 1) Making attack and image evaluators vectorized, so they can process batches efficiently. 2) Preprocess input data so that heavy work is only done once. 3) Automatically scale batch size based on the amount of vram the processing device has.

2.8.1 Application of High-Performance Computing

Due to the high demand for computation, DTU's High Performance Compute (HPC) platform was utilized. Both training and attacks were executed on the HPC, by applying an available NVIDIA A100 or V100 GPU depending on their availability.

3 Data

The pipeline ran through all combinations of the 9 models and 3 attack types resulting in a total of 27 combinations. This allowed for a smoother data collection as errors and lost data was minimized. A cutout of the final dataset for the base models can be seen in figure 5

	model	attack	true_class	target_class	original_pred_class	adversarial_pred_class	first_success_iter	attack_successful	psnr_score
0	mobilenet_imagenet20	cw	0	1	0	1	2.0	True	74.982132
1	mobilenet_imagenet20	cw	0	18	0	15	2.0	False	77.157402
2	mobilenet_imagenet20	cw	0	7	0	7	2.0	True	75.021255
3	mobilenet_imagenet20	cw	0	8	0	0	2.0	False	79.135536
4	mobilenet_imagenet20	cw	0	7	0	7	1.0	True	74.618340

ssim_score	ergas_score	adversarial_image_path	original_probs	adversarial_probs	dataset_index
0.999953	2486971.000	results/adversarial_images/adv_cw_src0_tgt1_id...	[0.9922124743461609, 0.00013835911522619426, 0...	[0.025619857013225555, 0.39093512296676636, 0....	0
0.999971	3537286.000	results/adversarial_images/adv_cw_src0_tgt18_i...	[0.9983968138694763, 4.403488492243923e-05, 1....	[0.17118822038173676, 0.022171422839164734, 0....	1
0.999955	3639137.000	results/adversarial_images/adv_cw_src0_tgt7_id...	[0.999265730381012, 1.2124605746066663e-05, 5....	[0.005473548546433449, 0.00010521702643018216,...	2
0.999982	2816849.500	results/adversarial_images/adv_cw_src0_tgt8_id...	[0.997897744178772, 0.0002516777894925326, 2.3...	[0.6428641080856323, 0.002640338847413659, 0.0...	3
0.999951	2692428.750	results/adversarial_images/adv_cw_src0_tgt7_id...	[0.9939563870429993, 0.00015116743452381343, 3...	[0.004608482588082552, 0.0003776552330236882, ...	4

Figure 5: Cutout of final dataset for base models

3.1 Model Accuracy

The test accuracies of each model variant on the test split of Imagenet20 are reported in Table 3. The three architectures overall achieve a very high baseline performance of 97.8%, 98.1% and 99.4% for MobileNet, ResNet and Swin, respectively.

Model	Baseline (Ref.)	ATM (% change)	REAM (% change)
Mobilenet	97.84	93.93 (–4.00)	82.70 (–15.48)
Resnet	98.09	97.05 (–1.06)	78.28 (–20.20)
Swin	99.44	96.72 (–2.71)	80.51 (–19.03)

Table 3: Test accuracies and relative changes by model and method

Adversarial training causes only a slight accuracy reduction with the largest drop being 4% for MobileNet, whereas robust training produces a much steeper decline across all architectures, peaking at a 20% drop for Resnet.

Table 4 provides a comparison of how the three architectures perform on average under the different training regimes.

Training Regime	Baseline (Ref.)	ATM (% change)	REAM (% change)
Average Accuracy	98.46	95.90 (–2.60)	80.50 (–18.24)

Table 4: Average test accuracy and relative changes across training regimes

3.2 Tests Availability

The data created by the pipeline is complex and difficult to apply statistical tests to as it violates some of the most basic rules of the most well known statistical tests such as ANOVA and Kruskal-Wallis. It is hierarchical and non-independent, and it can be grouped by the sample features as shown in Figure 6:

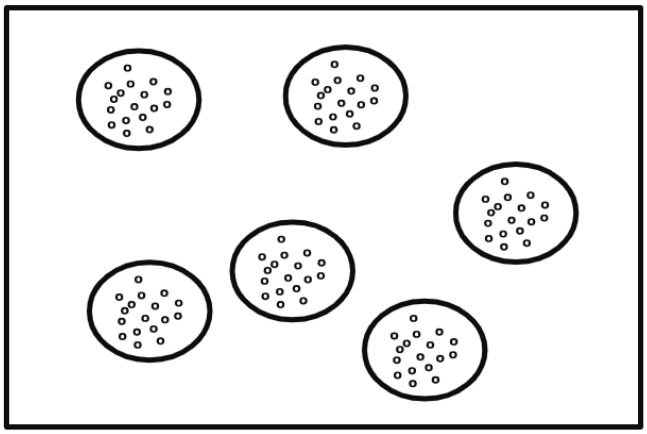


Figure 6: Illustration of hierarchical structure of data [45]

Here, the big circles indicate the individual images, where the smaller dots would be the repeated measures for each of the different attack types as well as for each of the target classes. Therefore, the hierarchical structure could actually be taken one step further and additionally split into subgroups in each of the big circles. The consequences of this dependency on the data is that it has been necessary to branch out and research other possible statistical tests.

3.3 Selected Test [Mixed-Effects Model]

One option to deal with hierarchical data to achieve independence could be narrowing focus to look at one circle at a time. Another option would be to average the samples in each circle and compare all six. Both solutions come with resulting cons, such as noisy outputs and not taking advantage of all available data. Linear mixed models exist as a trade-off between these two options and allow for exploring the different effects within and between groups.

At the core of mixed models is the incorporation of fixed and random effects. The mixed models are written as:

$$y = X\beta + Zu + \varepsilon$$

y is the outcome variable, X is a vector of the p predictor variables, β is a vector of fixed-effects regression coefficients, Z is the design vector for the random intercepts, u is a vector of the random effects and lastly ε is a vector of residuals explaining the part of y which is not represented by the model $X\beta + Zu$. The module `statsmodels.formula.api` is used for this statistical test as it includes function *mixedlm* for performing the mixed effects test [45].

4 Results

4.1 Base Models

Based on the dataset, key statistics were generated to describe and compare the 3 adversarial attacks on the base models. Table 5 displays the mean-values and their 95% confidence intervals for all attack types across evaluation metrics.

Attack	Target Class	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	9.61	6.41	81.10	85.84	0.9999964	7.9e+05
PGD	9.43	29.01	97.60	91.25	0.9999990	4.4e+05
CW	9.89	2.86	42.87	75.26	0.9999540	2.7e+06

(a) Adversarial attack metrics across different methods

Attack	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	[6.22, 6.60]	[0.80, 0.82]	[85.84, 85.85]	[1.0000, 1.0000]	[780259.50, 799836.00]
PGD	[28.67, 29.36]	[0.97, 0.98]	[91.23, 91.28]	[1.0000, 1.0000]	[429112.30, 440374.30]
CW	[2.83, 2.88]	[0.42, 0.44]	[75.23, 75.30]	[1.0000, 1.0000]	[2618388.90, 2688010.60]

(b) Confidence intervals for the metrics

Table 5: Performance metrics and confidence intervals for adversarial attack methods

First point of notice is that the adversarial class selected for the attacks lies around 9.5, matching the expectation (20 classes from 0-19, randomization should result in a mean of 9.5). It's immediately clear that there exists a difference between the attack types for the IQA scores PSNR and ERGAS, and a further look will therefore be necessary into these scores. In contrast, SSIM have very similar values with a standard deviation for all 3 attacks of less than 0.00003. Any statistical tests would not give any interesting insight into possible differences between the attack types. SSIM tries to calculate a numerical value which mimics the human visual system and such high values with little difference therefore indicate that the source image and perturbed image are extremely close to each other from a human point of view.

When observing first successful iteration of each attack, CW and I-FGSM appear significantly superior. However, a closer look at the ASR for each of the attacks reveals a very important aspect:

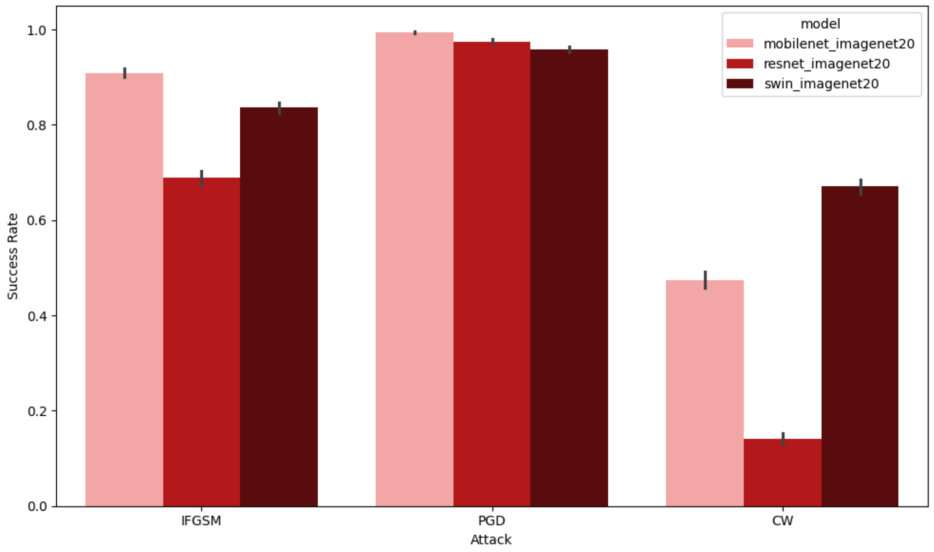


Figure 7: ASR by Attack and Model with 95% CI

ASR of PGD is the highest out of all three attack types, at 97.60%, with I-FGSM as a close second at 81.10%,

whereas CW has a significantly poorer ASR at only 42.87%, especially on ResNet-50 at 14.07%. This signifies that CW rarely succeeds compared to the other two attacks, but when it does, it needs considerably fewer iterations to achieve success.

4.1.1 Statistical Test

The PSNR- and ERGAS-score both display great indication of there being a significant difference between each model, which is explored in this section through the statistical test, mixed-effects model.

Significant difference in PSNR The mixed-effects models were conducted with the following parameters, where the variable being researched is the PSNR-score, the fixed effects are the model and attacks, and the random effect is the images.

Component	Value
Residual Variance (Scale)	1.42
Proportion of Variance Unexplained (%)	3
Dataset Index Variance (Random Effect)	0.34

Table 6: Variance Components from Mixed Effects Model

Intraclass Correlation Coefficient (ICC):

$$ICC = \frac{\sigma_{between}}{\sigma_{between} + \sigma_{within}} = \frac{0.343}{0.343 + 1.4203} = 0.195$$

The scale in this test is 1.4203, describing the variance of the dependent variable PSNR not explained by the fixed effect or random effect. The proportion of variance compared to the total variance is given in proportion unexplained, which in this case is 20%. A low number indicates that the mixed effects model fits the data well. The dataset index variance explains how much of the PSNR-score naturally varies across images, independent of model/attack differences. The ICC is low, at only 0.195, indicating images have little effect on the PSNR-score. Therefore, the effect is predominantly explained by the models and attacks used.

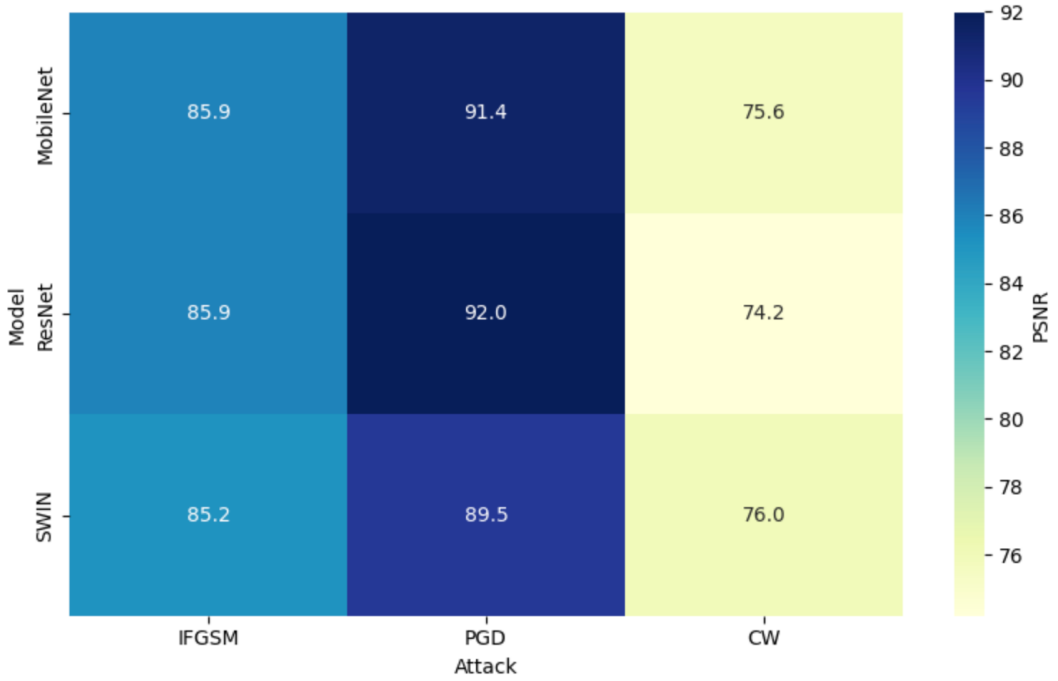


Figure 8: Expected PSNR by Model × Attack Type

Figure 8 displays the expected PSNR-score based on the model and the attack type. It illustrates how the PSNR-score is much higher across all models for PGD, with an average of 90.97%, compared to I-FGSM and CW, with

averages of 85.67% and 75.27%, respectively. From the mixed-effects test, all p-values approximate zero as the value was too low, which means that all combinations of models and attacks were significantly different from each other.

As PNSR is a measure of how much the perturbed image differs from the original image, this indicates that PGD is significantly better at keeping the perturbations to a minimum while still achieving a successful attack.

This conclusion is also supported in the Ergas score for the different attack types. A new mixed models test is conducted, where the PSNR score is replaced with the Ergas-score.

Significant difference in ERGAS
From Figure 9, it is now the expected ERGAS-score for each model and attack combination that can be seen. Here, a lower value is favorable as the score indicates the amount of difference between the original and perturbed image. Again, it is PGD that has the best score, followed by I-FGSM and then CW. Like with the PSNR-score test, all p-values are approximated to zero, and all combinations are significantly different from each other when it comes to performance. However, the significant difference is not the interesting part revealed by this test:

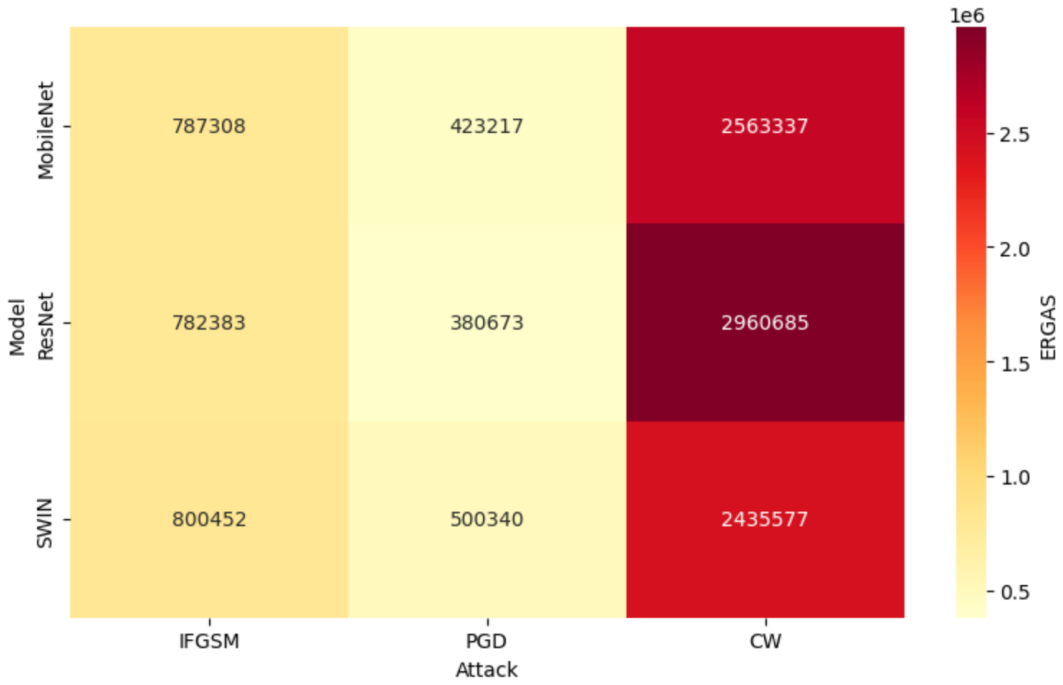


Figure 9: Expected ERGAS by Model × Attack Type

The scale and dataset index variance are stated below, and as can be seen, both the dataset index variances seem exceedingly high. The scale is compared to the total variance. Revealing 26% of the model to be unexplained by both the random intercepts and fixed effects, which means the model is a good fit for the data.

Component	**Value**
Residual Variance (Scale)	621623085254.42
Proportion of Variance Unexplained (%)	27
Dataset Index Variance (Random Effect)	733091266326.56

Table 7: Variance Components from Mixed Effects Model (ERGAS)

By using the dataset index variance and computing ICC, it gives a surprisingly high number:

$$ICC = \frac{733091266326}{733091266326 + 621623085254.4215} = 0.541$$

54% of the variance in the ERGAS-score comes from the image itself and not from the combination of model or attack type.

KL-divergence in probability distributions Another set of metrics returned from the pipeline was the probability distribution before and after the attack. A method of measuring adversarialness implemented here is by calculating the shift in the probability distribution through KL-divergence.

Table 8 shows the mean-values with standard deviations of KL-divergence for all 3 attack types:

	I-FGSM	**PGD**	**CW**
KL-divergence	3.80	7.56	2.11

Table 8: KL-divergence mean values for each attack type

From Table 8 it can be seen that PGD achieves the highest shift in probability distribution, whereas CW has the lowest. One is not necessarily better than the others, as a higher shift could equal a more severe attack, but a smaller shift is less detectable and still achieves the goal. Table 9 shows the correlation between KL-divergence and the PSNR score. The moderate connection implies that, on average, attacks which cause bigger shifts in the probability distribution also tend to apply larger image perturbations. This is not universal, as there are attacks that cause large shifts in probability with minimal distortion.

	KL-divergence	**PSNR-score**
KL-divergence	1.00	0.47
PSNR-score	0.47	1.00

Table 9: Correlation matrix between KL-divergence and PSNR-score

Figure 10 shows the estimated KL-divergence based on model and attack type and, as with the two previous tests, there seems to be a significant difference between the attack types, which is also supported by the p-values being approximated to zero. In this graph, there also appears to be bigger influence from the models as a all squares have values further from each other. As opposed to the graphs for PSNR and ERGAS above where the graph was more seperated into similar columns based on attacks.

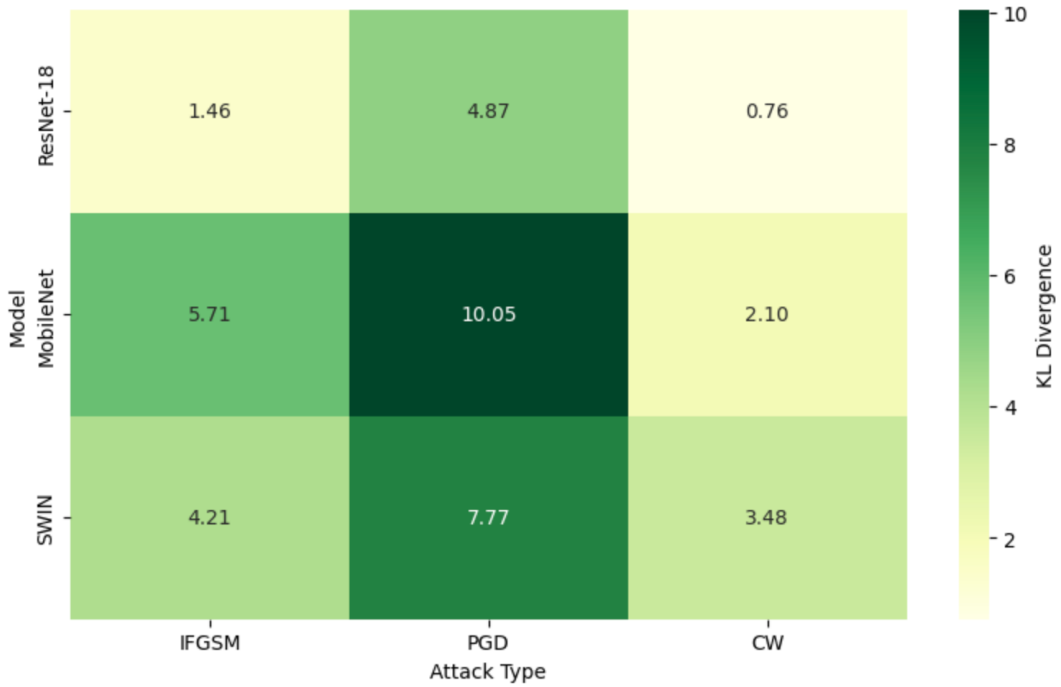


Figure 10: Estimated KL Divergence by Model × Attack

4.2 Architecturally Improved Models (REAM)

In addition to the base models, 3 architecturally improved models were created, where different methods were applied to improve the robustness based on the models' architecture. A comparison of the base vs robust models' general statistics can be seen in Table 10:

Attack	Adversarial Class	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	9.61	6.41	81.10	85.84	0.9999964	7.9e+05
PGD	9.43	29.01	97.60	91.25	0.9999990	4.4e+05
CW	9.89	2.86	42.87	75.26	0.9999540	2.7e+06
I-FGSM - REAM	9.34	3.90	55.71	85.30	0.999993	8.42e+05
PGD - REAM	9.33	51.25	59.19	86.76	0.9999950	7.13e+05
CW - REAM	8.88	4.39	22.34	72.67	0.999845	3.59e+06

(a) Adversarial attack metrics on base and robust models

Attack	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	[6.22, 6.60]	[80.39, 81.80]	[85.84, 85.85]	[1.00, 1.00]	[780259.53, 799836.04]
PGD	[28.67, 29.36]	[97.33, 97.88]	[91.23, 91.28]	[1.00, 1.00]	[429112.25, 440374.28]
CW	[2.83, 2.88]	[41.98, 43.76]	[75.23, 75.30]	[1.00, 1.00]	[2618388.95, 2688010.58]
I-FGSM - REAM	[3.70, 4.11]	[54.81, 56.60]	[85.29, 85.30]	[1.00, 1.00]	[831947.36, 852819.83]
PGD - REAM	[50.67, 51.84]	[58.30, 60.08]	[86.76, 86.77]	[1.00, 1.00]	[704092.86, 721737.10]
CW - REAM	[4.28, 4.50]	[21.59, 23.09]	[72.64, 72.70]	[1.00, 1.00]	[3546957.10, 3640138.65]

(b) Confidence intervals for metrics on base and robust models

Table 10: Performance and confidence intervals for adversarial attack methods on both base and robust models

PGD's mean first-success iteration increases from 29.01 to 51.25, and CW's from 2.86 to 4.39, indicating that on average both attacks require significantly more iterations to succeed. This is also reflected in the PSNR- and ERGAS-scores, which has decreased and increased respectively, as a result. The table displays the average ASR for the attack types across all 3 models, but in Figure 11 the ASR is explored a bit more:

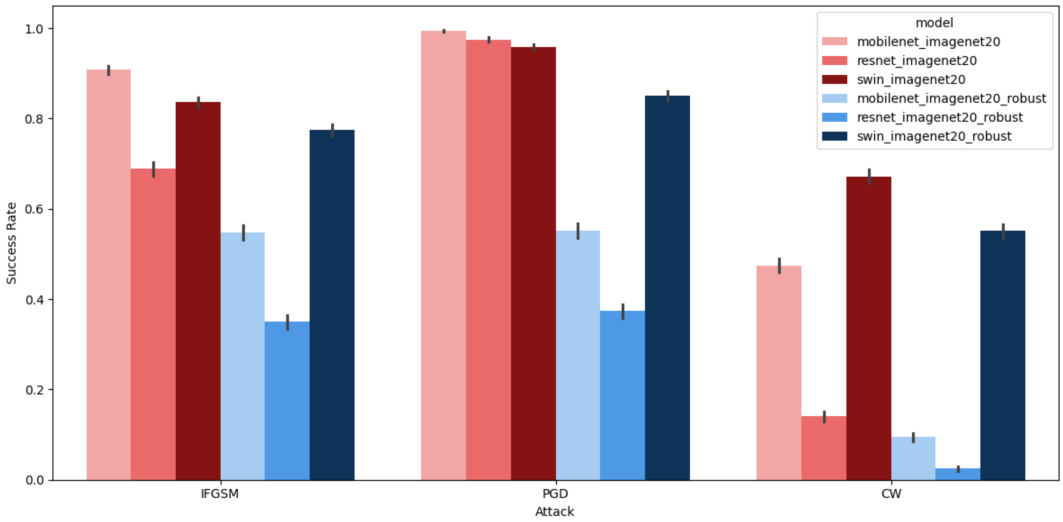


Figure 11: ASR by attack and Model with 95% CI

From the values and corresponding confidence intervals, it is obvious that there is a significant difference between ASR of different model and attack combinations. While the robustness methods applied to ResNet and MobileNet

cause a clear drop in success across all attack types, Swin also has a somewhat significant drop. From the visualization, there appears to be a slight difference between the attacks' effectiveness against the robustness measures, as CW seems to have a bigger reduction in success for both MobileNet and ResNet than I-FGSM and PGD have. I-FGSM appears to be the best at handling the robustness improved models, as it has the smallest drop in ASR with its updated ASR for ResNet and MobileNet ending at the same level as PGD's.

4.2.1 Statistical Test

To gain insight into the changes, the robustness methods introduced and the effect it had on the adversarial attacks performance table 10a compares the PSNR scores across the attacks on the base models versus the architectural robust models.

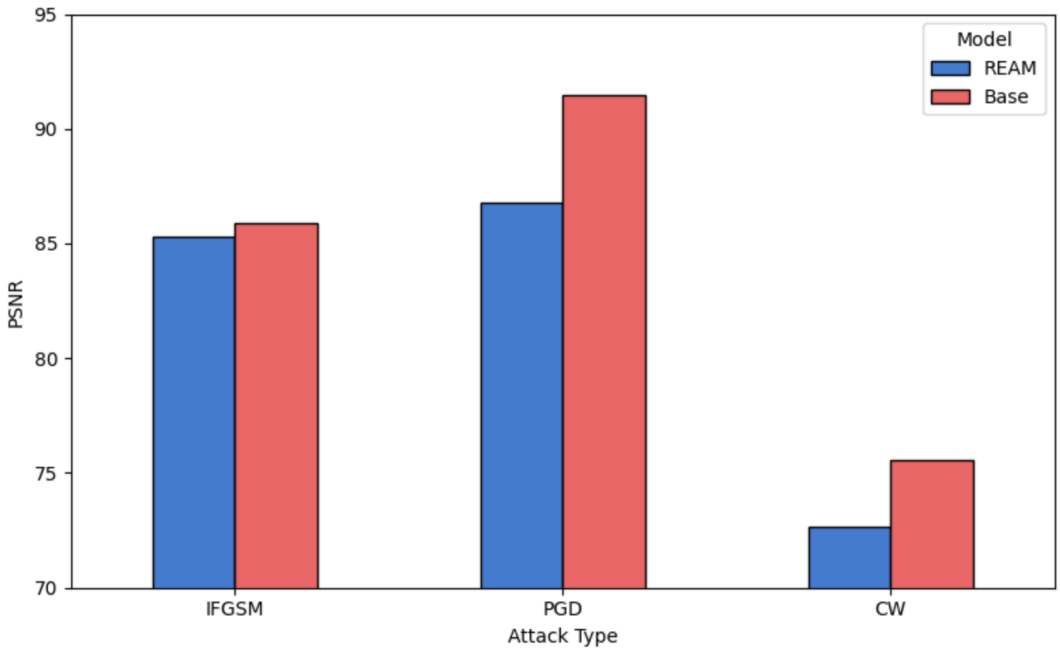


Figure 12: PSNR Comparison: Architectural vs Base Models

The PSNR-scores dropped for all 3 models, although only very slightly for I-FGSM, which means that all 3 attack types needed to apply more perturbations to the images in order to fool the models.

The mixed effects test is performed on the base and robust models to determine significant differences in PSNR between the model and attack type combinations:

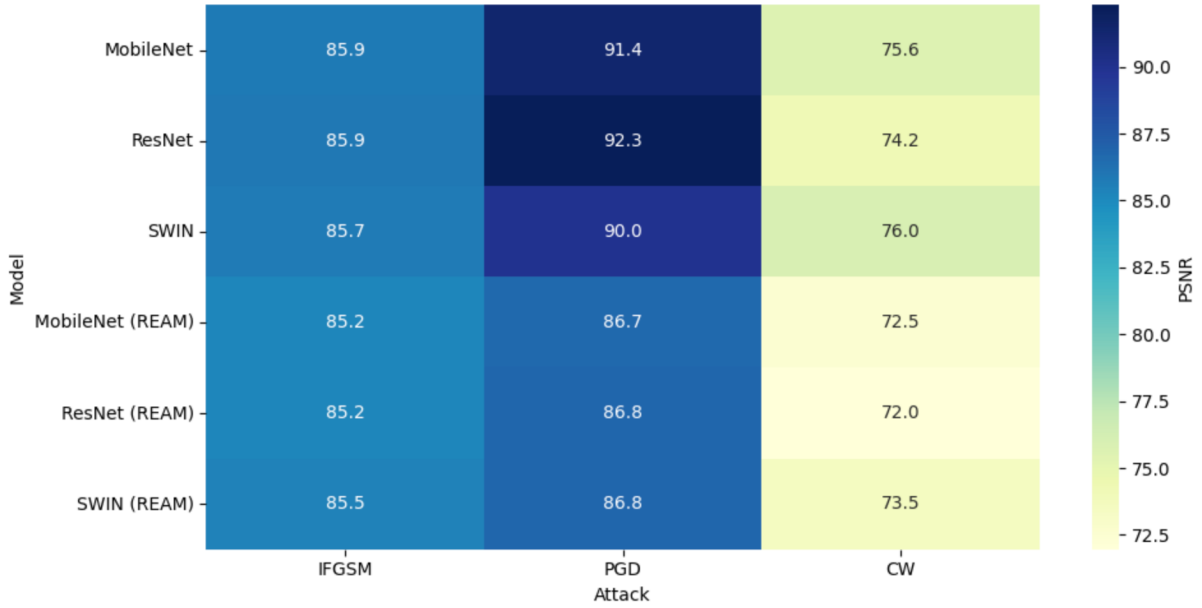


Figure 13: Expected PSNR by Model × Attack Type grouped by REAM vs Base models

The test revealed p-values approximating zero for each combination, meaning all were significantly different from each other. From the heatmap the reduction in the difference between PGD and I-FGSM can clearly be seen. Additionally it can seen that there is little difference in the expected perturbation applied by CW to the images between base and REAM.

Lastly the KL-divergence is calculated for adversarial attacks on the robust models and compared to the values from the base models:

Figure 14: Estimated KL Divergence by Model × Attack

There is a general reduction in the amount of shift present in the probability distributions before and after the attack except for the Swin model, which experience a slight increase for CW and I-FGSM. The KL divergence for PGD with the robust CNN based models shrinks down to nearly the same ranges as for I-FGSM.

4.3 Adversarially Trained Models

Another robustness improvement method used in this project is adversarial training. Three additional models were trained using adversarial examples, which brings the total model count to 9. The analysis conducted on the previous three robust models is repeated for the adversarial trained models.

4.3.1 Key Statistics

Table 11 shows an increase in the value of the first successful iteration for ATM compared to the base model and REAM. There is also a much greater decrease in the ASR for the adversarially trained models in relation to I-FGSM and PGD compared to the architectural models.

Attack	Adversarial Class	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	9.61	6.41	81.10	85.84	0.9999964	7.9e+05
PGD	9.43	29.01	97.60	91.25	0.9999990	4.4e+05
CW	9.89	2.86	42.87	75.26	0.9999540	2.7e+06
I-FGSM - REAM	9.34	3.90	55.71	85.30	0.999993	8.42e+05
PGD - REAM	9.33	51.25	59.19	86.76	0.9999950	4.89e+05
CW - REAM	8.88	4.39	22.34	72.67	0.999845	3.59e+06
I-FGSM - ATM	9.34	13.44	19.96	84.94	0.999993	8.80e+05
PGD - ATM	9.33	52.78	19.17	86.96	0.999995	7.11e+05
CW - ATM	9.26	10.50	19.33	73.45	0.999861	3.40e+06

(a) Adversarial attack metrics across different methods

Attack	First Success Iter.	ASR (%)	PSNR (dB)	SSIM	ERGAS
I-FGSM	[6.22, 6.60]	[80.39, 81.80]	[85.84, 85.85]	[1.00, 1.00]	[780259.53, 799836.04]
PGD	[28.67, 29.36]	[97.33, 97.88]	[91.23, 91.28]	[1.00, 1.00]	[429112.25, 440374.28]
CW	[2.83, 2.88]	[41.98, 43.76]	[75.23, 75.30]	[1.00, 1.00]	[2618388.95, 2688010.58]
I-FGSM - REAM	[3.70, 4.11]	[54.81, 56.60]	[85.29, 85.30]	[1.00, 1.00]	[831947.36, 852819.83]
PGD - REAM	[50.67, 51.84]	[58.30, 60.08]	[86.76, 86.77]	[1.00, 1.00]	[704092.86, 721737.10]
CW - REAM	[4.28, 4.50]	[21.59, 23.09]	[72.64, 72.70]	[1.00, 1.00]	[3546957.10, 3640138.65]
I-FGSM - ATM	[12.81, 14.07]	[19.24, 20.68]	[84.93, 84.95]	[1.00, 1.00]	[869285.52, 891265.26]
PGD - ATM	[51.77, 53.80]	[18.40, 19.81]	[86.93, 86.99]	[1.00, 1.00]	[701801.54, 720155.02]
CW - ATM	[10.22, 10.78]	[18.62, 20.05]	[73.40, 73.49]	[1.00, 1.00]	[3348211.96, 3442189.57]

(b) Confidence intervals for adversarial attack metrics across methods (rounded to two decimals)

Table 11: Performance and confidence intervals for adversarial attacks across base, REAM and ATM models

Figure 15 shows the ASR of the adversarial models compared to the base models and Figure 16 shows the ASR of all 9 models compared split into the three attack types. In general the adversarial training caused a much lower ASR on MobileNet and Swin, but it has little to no effect on ResNet. The architectural method used only had a small effect on the robustness of the Swin model, whereas adversarial training has resulted in a much greater reduction.

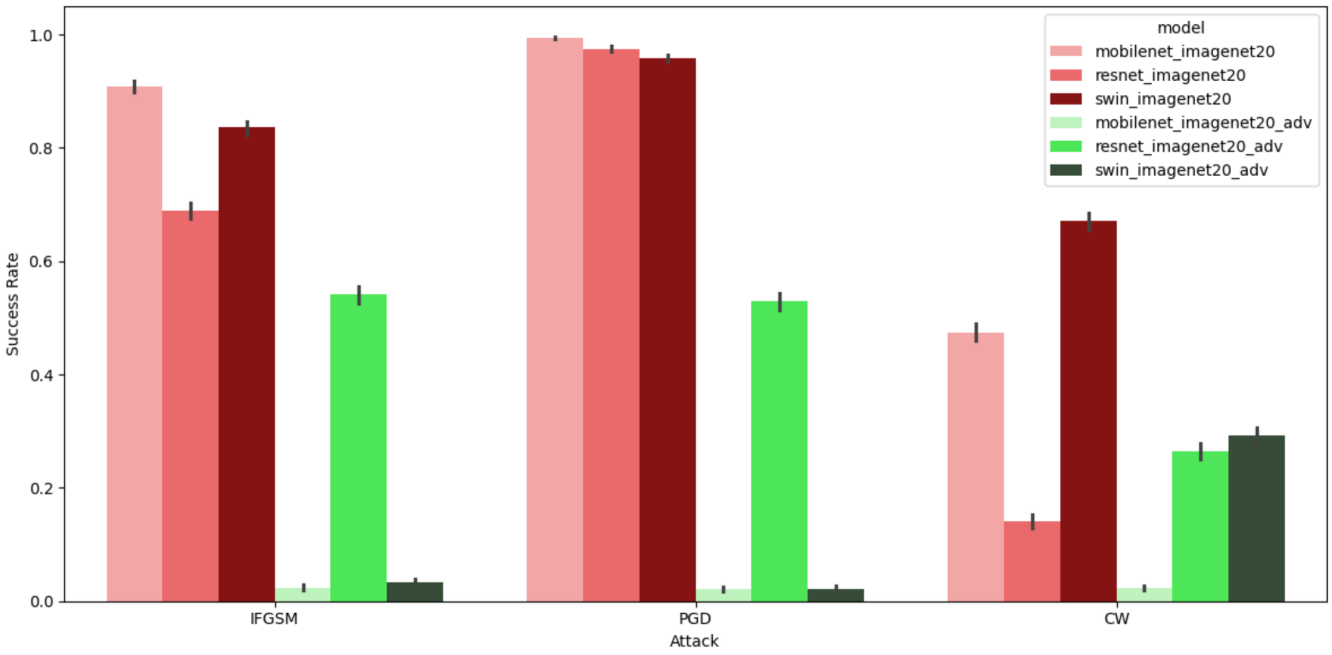


Figure 15: ASR by Attack and Model with 95% CI

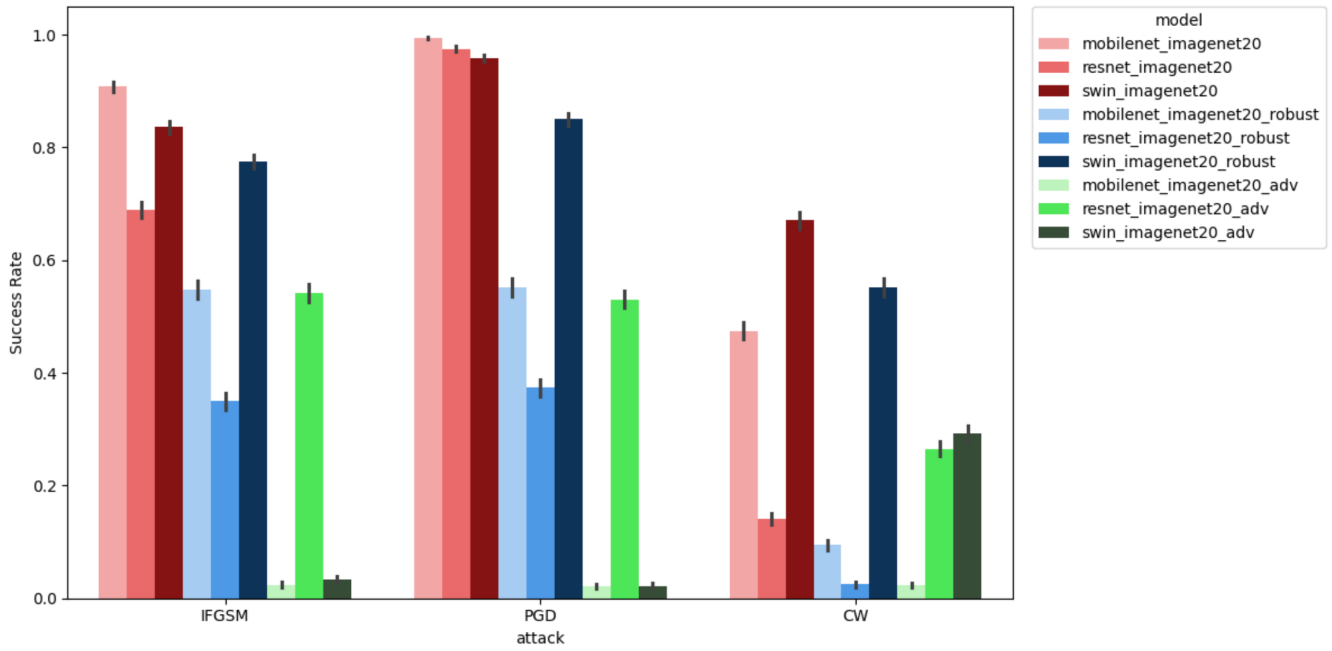


Figure 16: ASR by Attack and Model with 95% CI

4.3.2 Statistical Test

The statistical tests are expanded to include all 9 models. First, in Figure 17 the PSNR-score for all attack types averaged across the three models in each group is shown. There is a clear drop in the PSNR-score from the base models to the two types of robust models, but between the two types, there's only a small difference.

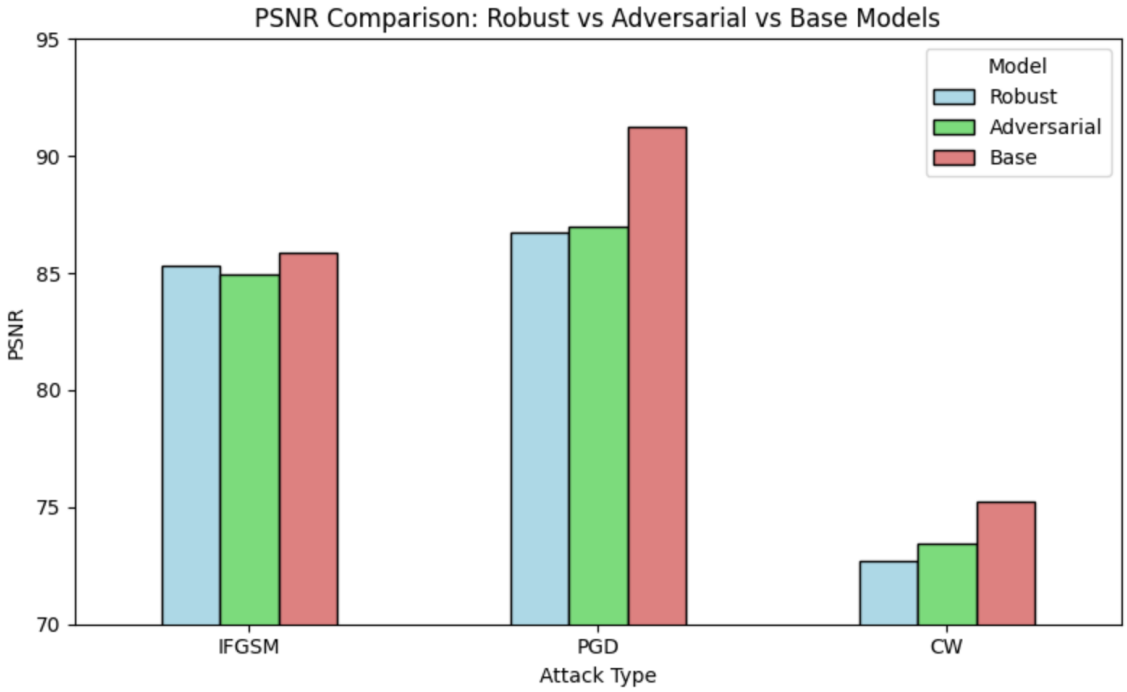


Figure 17: PSNR Comparison: REAM vs ATM vs Base Models

A mixed effects tests is conducted on the PSNR score for all 9 models and the results is illustrated in Figure 18. The clearest difference is between the expected values of PGD for the robust models and adversarial models compared to the base models. I-FGSM and CW have no distinct difference that stand out.

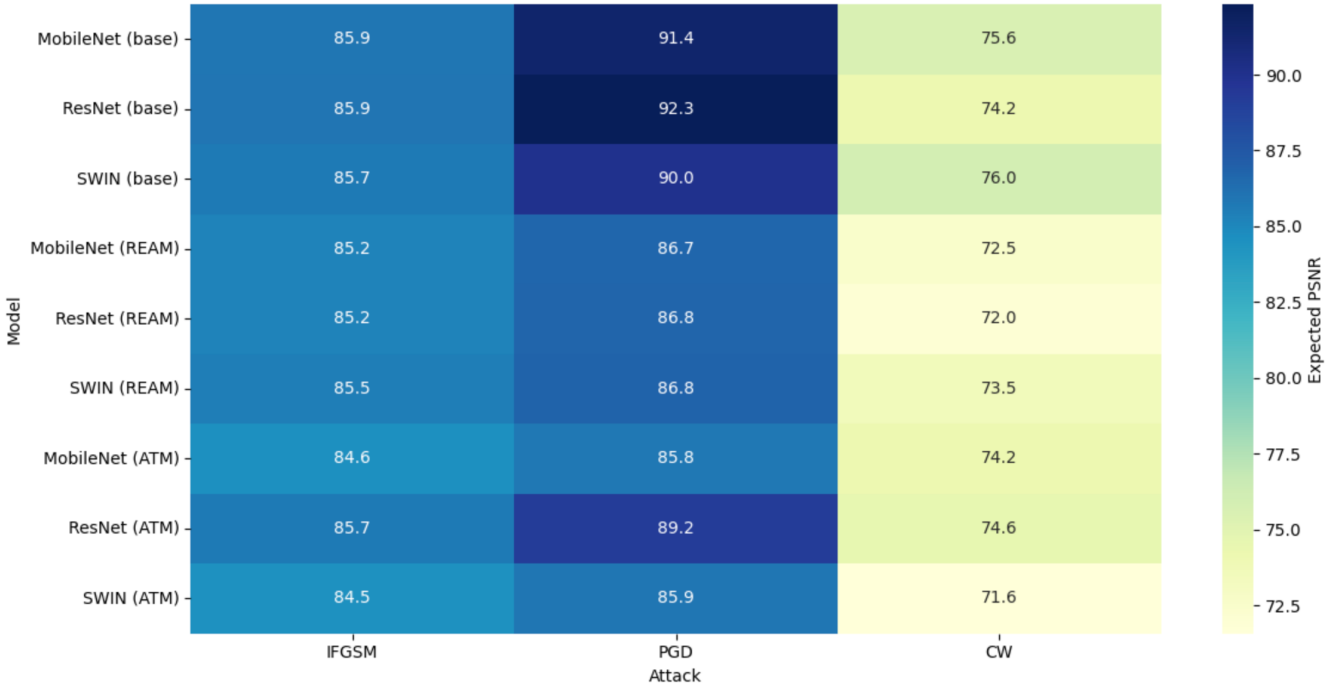


Figure 18: Heatmap over PSNR for all models

Lastly, the KL-divergence is computed and a mixed effects test is applied. Figure 19 shows a clear decrease in Kl-divergence between the adversarial trained models compared to the other 6 models, except for CW for Swin with adversarial training where a slight increase can be observed compared with the architectural robust model version. The largest difference can be seen in for PGD, that drops significantly with adversarial training.

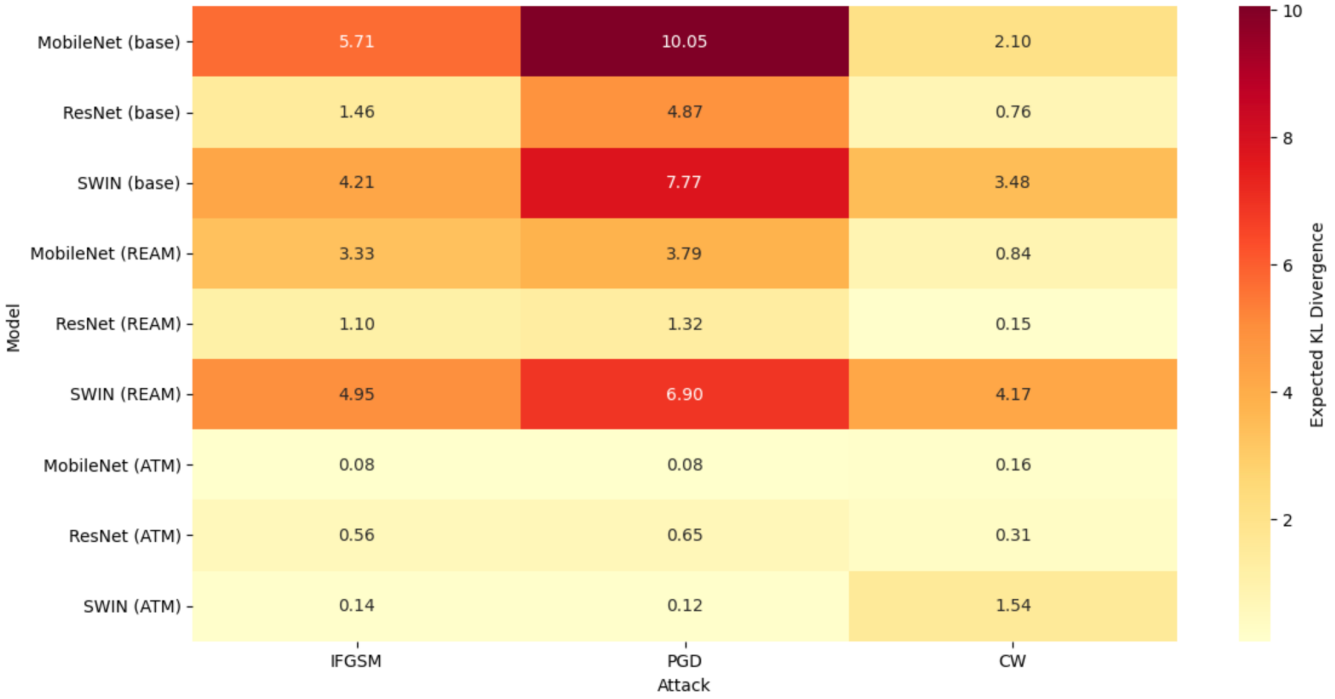


Figure 19: Heatmap over KL-divergence for all models

5 Discussion

Interpretation of results
The experiment conducted indicates that for our setup PGD is the best attack. CW has performed notably worse than expected, and I-FGSM has held up as a simpler yet reliable attack. The reasons for CW's lacking performance is most likely due to the environment, as the environment setup hasn't suited the method. There are two primary problems with the environment in regards to CW suboptimal performance. Firstly, 100 iterations is generally not considered enough. CW is expected to run around 1000 iterations, and additionally, the structure of its attacks is also much different than PGD and I-FGSM [4]. CW starts out by trying to hit the target class with no regard for the size of the perturbations added. After it has achieved misclassification, it then tries to minimize change, which is a much more complex and complicated process taking many more iterations. Therefore, the attack has not had the time to calibrate during the projects 100 iterations. Secondly, for both PGD and I-FGSM, there are commonly used parameter values for ϵ, α, etc. However, the parameters for CW are dynamic to the data and problem at hand, and search is recommended, which was not implemented for this project. This results in a less optimal application of the CW attack. For PGD and I-FGSM the results reflects the prior expectation of PGD performing better than I-FGSM as PGD is a continuation of I-FGSM.

The p-values for the difference between all combinations of models and attacks were approximated to be zero in the mixed effects test. This is actually to be expected as the sample size is so large that even small effects will appear as significant, which is a well documented phenomenon [46]. As multiple mixed effects tests will be conducted in this result section, a correction would be necessary to account for the multiple tests inflating the risk of type one errors. However, given the p-values approximating to zero due to a lack of computer precision a Bonferroni correction would be applied to p-values of zero. It would therefore still yield a significant result as the adjusted p-values would still approximate zero. For that reason, it was also not further interesting to look at the significance level as our primary source of difference between the attacks, but rather at their difference in ASR and the coefficients from the mixed effects tests, which was illustrated by the heatmaps.

The mixed effects statistical tests of the PSNR-score revealed CW as the attack type applying the most perturbations to the image, but given CW method as discussed above, this makes sense, as CW has not had time to minimize the difference between the source and perturbed image. Interestingly, the ERGAS test revealed 54% of the variance to be explained by the image used and not from the attack or model. This implies that some images have a similar effect across the attack types and models, where similar images prove to be more or less difficult for the attack to fool the model with. This might be due to some target classes generally being more difficult to achieve from the source image or some source images simply just being difficult to achieve success on, which then cause the images to have a much bigger impact on the ERGAS-score then the specific attack type.

REAM showed a significant drop in ASR due to the methods implemented in the models. The attacks on Swin had a slight decrease in ASR, but it was less distinct difference than MobileNet and ResNet experienced. Nothing in the data indicates that there should be any error source behind this, seemingly just a case of the robustness technique selected for Swin not being as effective as the ones selected for MobileNet and ResNet.

I-FGSM was surprisingly not particularly affected by the robustness techniques compared to PGD and CW, with its ASR ending around the same level as PGD despite it being a simpler attack. However, this is due to the effect that the selected robustness method relies on, as I-FGSM is less affected by a smoother gradient, making it appear on par with PGD. The gradients affects PGD and CW more than it does I-FGSM. This makes I-FGSM appear relatively better compared to PGD and CW then before, although it still overall experiences a drop in ASR.

It was also PGD and CW that had the biggest average change in PSNR score, where there was little difference in I-FGSM. The smoother gradient minimizes the benefit of PGD's random perturbation start, causing it to use the full perturbation budget just like I-FGSM. CW firstly tries to achieve misclassification and then to minimize the amount of perturbation, but when the gradient is smooth, the attack is force to take more steps to before misclassification, which in turn leads to more noise and lower PSNR.

ATM showed a great decrease in ASR for MobileNet and Swin, far greater than that for the REAM but it had little effect on ResNet. This is a documented phenomenon as skip connections and convolutional biases make adversarial training less effective on ResNet [47]. For ResNet, Squeeze & Excitation was preferred as it reduced the impact of the vulnerability in the models architecture. In contrast, the adversarial training had a much better effect for the Swin model's robustness as it was much more effective at withstanding the attacks.

Despite the continued high ASR on ResNet, the KL-divergence for the adversarially trained models across the board,

were all much smaller than both the base models and architectural models. So, while the attacks still succeed at making ResNet misclassify, they do not achieve as great a general shift in the models probability distribution.

An important perspective on ATM versus REAM is the effectiveness against new attacks. Adversarial training trains the models to withstand specific attack types, but if the model was exposed to new attacks, the adversarial training would have minimal effect as its generalization is limited [48]. Contrastingly, more robust architectures do not depend on the specific attack type and generalize better across all attack types providing a broader protection against adversarial attacks [49]. Additionally, in the data section, the accuracy of all 9 models on test data was presented. Here, it could be clearly seen that the architectural improvements cause a bigger drop in successful classification of images. There is a sacrifice of accuracy in order to protect the models from adversarial attacks. ATM did not experience as big of a drop. REAM might provide a better generalized protection against adversarial attacks, but it comes at the cost of accuracy. Depending on the field where the models are to be applied, this can affect whether or not this trade-off is attractive and needed or not.

A key factor for enabling comparisons of the attack types was the selection of the quantification methods. For this project, three types of quantification was used, which was ASR, IQA methods, and KL-divergence in probability distribution. ASR and IQA are central methods for measuring the severity of attacks, whereas KL-divergence is more recognized in robustness-improving perspectives. For the IQA methods, this project utilized three methods, which were PSNR, SSIM, and ERGAS. The SSIM-score for the samples were so close to each other and to one that any further comparison would be redundant, as it would highlight an insignificant difference. The most important take-away from this score was the values almost being one, indicating that the perturbed images are approximately undetectable for the human eye. While the quantification methods used in this project are still highly relevant and used across present experiments and reports, new methods have been presented during the last year, which were discovered in the literature late in the process of this project. These methods include Adversarial hypervolume for quantifications of adversarialness, which is a continuation of ASR for capturing success rate across epsilon budgets. Another is ACTS, which quantify robustness per input by measuring how many gradient-based update steps are required for an adversarial attack to succeed. Incorporating these metrics in future work could yield deeper insights into model behavior across perturbation strengths and guide the development of more uniformly robust defenses.

Project design and limitations

The intended outcome of this study was to gain a better understanding of how attack effectiveness varies across models and robustness improvements. However, is this even a fair thing to do? In our experiment the source image and target classes has been kept constant for all model and attack combinations, which is what allows for comparison of the attacks. However, the ERGAS-score indicates that the source image has a large effect on the level of distortion applied by of all of the attack types. A point of notice for future works would be to look into the effect source and target class has on the adversarial attacks.

All pre-trained ImageNet-1K weights were loaded, only retraining the head to change the output dimension to 20 classes. For ATM, we further finetuned the weights with attack examples to improve robustness. However, this was not possible to do for REAM due to the architectural changes. They were, instead, trained from the ground up, which may contribute to their worse performance. Furthermore, the robust models were only trained on a 20-class dataset, giving them less contextual depth and understanding, limiting their grasp. This contributed to an average drop of 18.24% (see Table 3). This raises the question of whether differences in accuracy stem from robustness-focused architectures or simply from training setup, complicating direct comparisons across methods.

For many parts of the project, decisions have been made that could have been different. Values chosen, choices taken/not taken, and these all accumulate to an uncertainty in their importance on the results. Another example of this is our setup of adversarial training, which has been made with random sampling as to how often an attack was used for finetuning, compared to just the source-image (with an 80-20 split). Furthermore, there are considerations of balancing distortion: meaning, which attacks do we "fear" the most? This balance could also be adjusted, whereby we simply used a 50-50 split between either I-FGSM or PGD. If we assumed PGD to be a more dangerous attack, perhaps weighing training against it would be a more robust approach.

Considering fundamental choices, what others could have been made? A large difference is the choice of the attacks being targeted. This choice was made due to the real-world application, i.e. the example of changing traffic signs to specific "dangerous" ones. We thought that the control of the attack was important, and we wished to study this. However, for many other applications, it may be better to focus on untargeted attacks; for example, in spam filtering, simply causing any spam email to be misclassified as legitimate mail is sufficient. Different model architectures could also have been explored, such as decision trees or other classifier types that are less susceptible

to adversarial manipulation. These do not feature a loss function and thereby, stand a better chance against gradient-based attacks. Choosing different robustness methods for REAM was based on the fact that we couldn't use a standardized version for all, as they had fundamentally differing architectures. However, this is again also creating a more distorted image of the outcomes - as the change from base to REAM may not be the same across all three models.

Ethical Perspective and Considerations

Our society is rapidly adopting machine learning into daily decision-making, introducing previously unseen risks related to adversarial attacks. Consider a scenario with a self-driving car. The car has several different inputs to interpret its surroundings with. Given there exists a neural network that interprets the surroundings to make just decisions, the network can be attacked. One example might be a vandal who put a sticker onto a "kids playing" sign - making the neural network and sole decision-maker of your car observe this sign as a "highway coming up!" sign. Due to the complete trust put into the car's superior decision-making, such vulnerabilities start to exist - in turn creating new problems. For the attacks conducted in this project, which are based on CNN/transformer classifiers and their processing of 224-by-224 pixels imagery, applying the loss-function to carefully manipulate the output, the self-driving car example still illustrates the same principle; increasing reliance on machine learning models opens up new vulnerabilities and dangers.

Data formats that are directly interpreted by a model, and not changed (assuming no compression), pose the real threat for attacks conducted in this project. Humans have no realistic chance of observing nor detecting, shown by SSIM-scores of near-1 for all attacks (see Table 11), whether an image has been manipulated and/or is an attack. One scenario where direct images are used is with AI systems in the medical field. It has proven time and time again to have great effectiveness at detection and diagnosis [50], which poses huge improvements for SDG 3, "*Good Health and Well-being*". However, if a bad actor were to systematically impact the image processing, it could have massive real-world consequences. For most medical practice, targeting specific groups has been difficult due to the sheer size. However, one could create attacks on the detection of terminal illnesses, only applied for a targeted group, posing a threat to SDG 10, "*Reduced Inequalities*". Much of medical decisions have previously been based on multiple opinions–if you don't trust it, ask a different doctor–however, if this different doctor now deploys the same model, the wrong diagnoses continue. Although these may not seem like realistic outcomes of AI innovation, the truth lingers within these arguments: The loss of human control over AI-driven decision-making and the possibility that this control falls into the hands of adversarial attacks, could lead to significant public concern.

Doctors have always made mistakes, and continue to do so. This is accepted and understood as a result of trial and error. People trust that the doctor acts in good faith, and wants the best for them. Their acceptance and comfort in these facts do not translate directly over to their understanding of the role AI has, where "*it was wrong, but they tried their best*" - assessments dissipate. The decisions of these models are not based in good faith, rather based on weighing the probability of an outcome based on the data it has been trained on. Although AI systems diminish bias, the danger of abusing these systems greatly increases.

ResNet, ImageNet, and Swin are among some of the most popular open-sourced models for image classification, advocated as a solution for businesses [51], which many use. As a business, it's a great solution, as the models are easy to apply, well-documented and trustworthy. However, this openness poses real dangers if you have sensitive data or any way to abuse the system. These white-box models, if applied to sensitive tasks, may become a dangerous solution.

On balancing robustness and accuracy: One might argue for balancing between adversarial robustness and accuracy, but there is no true balance - the balance is dependent on the task at hand.

5.1 Future works

While our current study offers insight into attack effectiveness across models and defenses, several changes remain to be explored:

- **Stronger CW evaluation:** Increase CW to 1000 iterations and implement a dynamic search for its regularization parameter to reach its full potential.
- **Hyperparameter sweeps:** Systematically vary ϵ, α, attack–training splits and the mix of attack types in adversarial fine-tuning.
- **Source-to-target labels:** Quantify class-to-class distances to predict which transitions are inherently harder to achieve successful misclassifications.
- **Untargeted attacks:** Extend experiments to untargeted methods to see if trends hold when the attacker's goal is simply misclassification, not a specific label.
- **Alternative architectures and defenses:** Evaluate non-CNN and transformers classifiers and other robustness techniques.
- **Alternative dataset:** Apply attacks across different datasets to evaluate the importance of image-specific robustness
- **Fairness in comparison:** Standardize pretraining and training protocols across all model variants to isolate the effect of defense method from training setup.
- **Generalization to novel attacks:** Test each defense against attacks not seen during training to gauge true robustness beyond the trained threat model.
- **Real-world simulation:** Further exploration should be done without the computational constraints to mimic the real world application where attack performance is the only consideration.

6 Conclusion

This project aims to evaluate and compare the severity of different adversarial attacks, I-FGSM, PGD and CW, across three popular open-sourced model architectures, MobileNet, ResNet and Swin. We tested on both the original models and variations with two distinct robustness strategies, adversarial training (ATM) and architectural changes (REAM) applied.

The conducted experiments reveal PGD as the most effective attack, with the highest ASR and least distortive perturbations, while CW underperformed due to limited tuning and iteration constraints. For defenses, ATM outperformed REAM - whereof both significantly reduced ASR and KL-divergence metrics. Surprisingly, due to model architecture, REAM ResNet performed superior to ATM ResNet. For both robustness methods, the underlying trade-off between robustness and generalization was revealed, where a loss in accuracy was always the cost of improving robustness. Furthermore, our mixed-effects analysis revealed ERGAS to have a large amount of variance explained by the images themselves, suggesting fundamental differences in the difficulty of creating an adversarial example across images. These findings show the necessity for thorough defenses when deploying deep learning models safely, especially for sensitive domains.

The differences in effectiveness of attack and model combinations are highlighted, namely the under-performance of CW, due to mismatch of parameters for optimal performance. Furthermore, the different training setups and the uncertainty created with this is also discussed, which in-turn creates dangers between comparing ATM and REAM. Future work should thoroughly optimize and test CW attacks across models. Another suggestion is attempting attacks across different datasets to evaluate the importance of image-specific robustness, or complete a closer evaluation of robustness transferability between attacks.

In summary: PGD emerged as the most effective adversarial attack, and even then, when incorporating carefully selected defenses for a given model, it was able to be countered effectively–although at a price which one must weigh themselves.

Bibliography

[1] Rakesh Podder and Sudipto Ghosh. *Impact of White-Box Adversarial Attacks on Convolutional Neural Networks*. 2024. arXiv: 2410.02043 [cs.CR]. URL: https://arxiv.org/abs/2410.02043.

[2] Samy Bengio Alexey Kurakin Ian J. Goodfellow. "ADVERSARIAL EXAMPLES IN THE PHYSICAL WORLD". In: *Computer Vision and Pattern Recognition*. 2017. URL: https://arxiv.org/pdf/1607.02533.

[3] Aleksander Madry et al. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2019. arXiv: 1706.06083 [stat.ML]. URL: https://arxiv.org/abs/1706.06083.

[4] N. Carlini and D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. (accessed: 10.06.2025). URL: https://arxiv.org/pdf/1608.04644.

[5] Sid Ahmed Fezza et al. *Perceptual Evaluation of Adversarial Attacks for CNN-based Image Classification*. 2019. arXiv: 1906.00204 [cs.LG]. URL: https://arxiv.org/abs/1906.00204.

[6] Jing Wu et al. *Performance Evaluation of Adversarial Attacks: Discrepancies and Solutions*. 2021. arXiv: 2104.11103 [cs.LG]. URL: https://arxiv.org/abs/2104.11103.

[7] Baoyuan Wu et al. *Defenses in Adversarial Machine Learning: A Survey*. 2023. arXiv: 2312.08890 [cs.CV]. URL: https://arxiv.org/abs/2312.08890.

[8] Jeremy M Cohen, Elan Rosenfeld, and J. Zico Kolter. *Certified Adversarial Robustness via Randomized Smoothing*. 2019. arXiv: 1902.02918 [cs.LG]. URL: https://arxiv.org/abs/1902.02918.

[9] Cihang Xie et al. *Feature Denoising for Improving Adversarial Robustness*. 2019. arXiv: 1812.03411 [cs.CV]. URL: https://arxiv.org/abs/1812.03411.

[10] Shruthi Gowda, Bahram Zonooz, and Elahe Arani. *Conserve-Update-Revise to Cure Generalization and Robustness Trade-off in Adversarial Training*. 2024. arXiv: 2401.14948 [cs.LG]. URL: https://arxiv.org/abs/2401.14948.

[11] Leo Hyun Park et al. *Adversarial Feature Alignment: Balancing Robustness and Accuracy in Deep Learning via Adversarial Training*. 2024. arXiv: 2402.12187 [cs.CV]. URL: https://arxiv.org/abs/2402.12187.

[12] Jindong Gu et al. *A Survey on Transferability of Adversarial Examples across Deep Neural Networks*. 2024. arXiv: 2310.17626 [cs.CV]. URL: https://arxiv.org/abs/2310.17626.

[13] Lumenova AI. *Types of Adversarial Attacks and How To Overcome Them*. (accessed: 09.06.2025). URL: https://www.lumenova.ai/blog/overcoming-adversarial-attacks-machine-learning/.

[14] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. "Explaining and Harnessing Adversarial Examples". In: *International Conference on Learning Representations (ICLR)*. Available at: urlhttps://arxiv.org/pdf/1412.6572. 2015. URL: https://arxiv.org/abs/1412.6572.

[15] Rishika Bhagwatkar et al. *Towards Adversarially Robust Vision-Language Models: Insights from Design Choices and Prompt Formatting Techniques*. 2024. arXiv: 2407.11121 [cs.CV]. URL: https://arxiv.org/abs/2407.11121.

[16] Arun George Zachariah. *Adversarial Attacks with Carlini & Wagner Approach*. (accessed: 30.03.2025). URL: https://medium.com/@zachariaharungeorge/adversarial-attacks-with-carlini-wagner-approach-8307daa9a503.

[17] 2018. URL: https://adversarial-robustness-toolbox.readthedocs.io/en/latest/modules/attacks/evasion.html?utm_source=chatgpt.com#carlini-and-wagner-l-0-attack.

[18] Stanford Vision Lab, Stanford University, and Princeton University. *imagenet downloader*. (accessed: 09.06.2025). URL: https://www.image-net.org/download.php.

[19] Wei Dong Jia Deng and Richard Socher. 2019. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=5206848&tag=1.

[20] Yonglong Tian, Dilip Krishnan, and Phillip Isola. *Contrastive Multiview Coding*. 2020. arXiv: 1906.05849 [cs.CV]. URL: https://arxiv.org/abs/1906.05849.

[21] Ze Liu et al. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. 2021. arXiv: 2103.14030 [cs.CV]. URL: https://arxiv.org/abs/2103.14030.

[22] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

[23] Andrew G. Howard et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. 2017. arXiv: 1704.04861 [cs.CV]. URL: https://arxiv.org/abs/1704.04861.

[24] Norah Al-Humaidan and Master Prince. "A Classification of Arab Ethnicity Based on Face Image Using Deep Learning Approach". In: *IEEE Access* PP (Mar. 2021), pp. 1–1. DOI: 10.1109/ACCESS.2021.3069022.

[25] Pierre Foret et al. *Sharpness-Aware Minimization for Efficiently Improving Generalization*. 2021. arXiv: 2010.01412 [cs.LG]. URL: https://arxiv.org/abs/2010.01412.

[26] Jie Hu et al. *Squeeze-and-Excitation Networks*. 2019. arXiv: 1709.01507 [cs.CV]. URL: https://arxiv.org/abs/1709.01507.

[27] Aleksander Madry et al. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2019. arXiv: 1706.06083 [stat.ML]. URL: https://arxiv.org/abs/1706.06083.

[28] Masoud Daneshtalab Hamid Mousavi Ali Zoljodi. *Analysing robustness of tiny deep neural networks*. URL: https://www.es.mdu.se/pdf_publications/6711.pdf.

[29] Janani Suresh, Nancy Nayak, and Sheetal Kalyani. *First line of defense: A robust first layer mitigates adversarial attacks*. 2024. arXiv: 2408.11680 [cs.LG]. URL: https://arxiv.org/abs/2408.11680.

[30] Sana Hassan and Sana Hassan. *Exploring Robustness: Large Kernel ConvNets in Comparison to Convolutional Neural Network CNNs and Vision Transformers ViTs*. 2024. URL: https://www.marktechpost.com/2024/07/15/exploring-robustness-large-kernel-convnets-in-comparison-to-convolutional-neural-network-cnns-and-vision-transformers-vits/.

[31] * Kriuk Fedor Kriuk Boris and Praveen Karthik. *Smooth Attention: Improving Image Semantic Segmentation*. 2024. URL: https://journalspress.com/LJRCST_Volume24/Smooth-Attention-Improving-Image-Semantic-Segmentation.pdf.

[32] Xuechen Zhang et al. *Selective Attention: Enhancing Transformer through Principled Context Control*. 2024. arXiv: 2411.12892 [cs.LG]. URL: https://arxiv.org/abs/2411.12892.

[33] Shuangfei Zhai et al. *Stabilizing Transformer Training by Preventing Attention Entropy Collapse*. Proceedings, 2023. URL: https://proceedings.mlr.press/v202/zhai23a/zhai23a.pdf.

[34] Chuan Guo et al. *On Calibration of Modern Neural Networks*. 2017. arXiv: 1706.04599 [cs.LG]. URL: https://arxiv.org/abs/1706.04599.

[35] 2017. URL: https://docs.aws.amazon.com/prescriptive-guidance/latest/ml-quantifying-uncertainty/temp-scaling.html.

[36] Hao Xuan, Bokai Yang, and Xingyu Li. *Exploring the Impact of Temperature Scaling in Softmax for Classification and Adversarial Robustness*. 2025. arXiv: 2502.20604 [cs.LG]. URL: https://arxiv.org/abs/2502.20604.

[37] Xuanyi Wu et al. "RAVA: Region-Based Average Video Quality Assessment". In: *Sensors* 21.16 (2021), pp. 5489–5489. DOI: https://doi.org/10.3390/s21165489. URL: https://www.mdpi.com/1424-8220/21/16/5489.

[38] 2024. URL: https://www.ni.com/en/shop/data-acquisition-and-control/add-ons-for-data-acquisition-and-control/what-is-vision-development-module/peak-signal-to-noise-ratio-as-an-image-quality-metric.html.

[39] D. Sethi and Bharti. *A comprehensive survey on gait analysis: History, parameters, approaches, pose estimation, and future work*. (accessed: 20.06.2025). URL: https://www.sciencedirect.com/science/article/pii/S0933365722000793?ref=pdf_download&fr=RR-2&rr=9529d3c9ce58be42.

[40] Jim Nilsson and Tomas Akenine-Möller. *Understanding SSIM*. 2020. arXiv: 2006.13846 [eess.IV]. URL: https://arxiv.org/abs/2006.13846.

[41] Alan Conrad Bovik Zhou Wang Member and Hamid Rahim Sheikh. *Image Quality Assessment: From Error Visibility to Structural Similarity*. 2004. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1284395&tag=1.

[42] H. R. Sheikh Zhou Wang A. C. Bovik and E. P. Simoncelli. *Image Quality Assessment: From Error Visibility to Structural Similarity*. 2004. URL: https://doi.org/10.1109/tip.2003.819861.

[43] torchmetrics. *Error Relative Global Dim. Synthesis (ERGAS)*. (accessed: 10.06.2025). URL: https://torchmetrics.readthedocs.io/en/v0.9.3/image/error_relative_global_dimensionless_synthesis.html.

[44] Lightning AI. *Error Relative Global Dim. Synthesis (ERGAS)*. (accessed: 10.06.2025). URL: https://lightning.ai/docs/torchmetrics/stable/image/error_relative_global_dimensionless_synthesis.html.

[45] UCLA. *How to Evaluate Image Quality in Python: A Comprehensive Guide*. URL: https://stats.oarc.ucla.edu/other/mult-pkg/%20introduction-to-linear-mixed-models/.

[46] Gerd Gigerenzer, Stefan Krauss, and Oliver Vitouch. "The Null Ritual: What You Always Wanted to Know About Significance Testing but Were Afraid to Ask". In: (Jan. 2004).

[47] Dongxian Wu et al. "Skip Connections Matter: On the Transferability of Adversarial Examples Generated with ResNets". In: *CoRR* abs/2002.05990 (2020). arXiv: 2002.05990. URL: https://arxiv.org/abs/2002.05990.

[48] Florian Tramèr and Dan Boneh. *Adversarial Training and Robustness for Multiple Perturbations*. 2019. arXiv: 1904.13000 [cs.LG]. URL: https://arxiv.org/abs/1904.13000.

[49] Naman D Singh, Francesco Croce, and Matthias Hein. *Revisiting Adversarial Training for ImageNet: Architectures, Training and Generalization across Threat Models*. 2023. arXiv: 2303.01870 [cs.CV]. URL: https://arxiv.org/abs/2303.01870.

[50] Janet Wang et al. *Doctor Approved: Generating Medically Accurate Skin Disease Images through AI-Expert Feedback*. 2025. arXiv: 2506.12323 [cs.CV]. URL: https://arxiv.org/abs/2506.12323.

[51] Rearc. *Image Recognition with PyTorch Resnet*. (accessed: 20.06.2025). URL: https://aws.amazon.com/marketplace/pp/prodview-2fcetnqpm6hlw.

A Appendix

A.1 Project Repository

The code used in our study, including script for training model, attacks and IQA implementations and the experiment pipeline. The repository provides all necessary code and environment desciption to replicate our experiment and analyses.

Github repository link: https://github.com/murrodroid/AdversarialAttacks

A.2 ImageNet100 Dataset Link

Link to ImageNet100: https://huggingface.co/datasets/clane9/imagenet-100